

Explicación del Desarrollo del Software Realizado (Sistema, Aplicación)

El sistema desarrollado es una aplicación web para la gestión de citas médicas y perfiles de usuarios y médicos para el Instituto de Seguridad y Servicios Sociales de los Trabajadores del Estado (ISSSTE). La aplicación permite a los usuarios afiliados agendar citas, ver su historial de citas y crear opiniones sobre los médicos. A su vez, los médicos pueden gestionar sus perfiles, registrar informes médicos y consultar su calendario de citas. La aplicación está construida en Java utilizando la biblioteca Swing para la interfaz gráfica de usuario (GUI) y se conecta a una base de datos MySQL para el almacenamiento de datos.

Pantallas Principales del Desarrollo

Pantallas de Usuario Afiliado

1. **Inicio**
 - Muestra la misión y visión del ISSSTE.
 - Proporciona información de contacto.
2. **Equipo Médico**
 - Permite a los usuarios ver los médicos disponibles por especialidad.
 - Incluye una sección de opiniones de otros usuarios sobre los médicos.
3. **Preguntas Frecuentes**
 - Presenta una lista de preguntas comunes con sus respuestas.
 - Se alimenta de un HashMap que contiene las preguntas y respuestas predefinidas.
4. **Perfil**
 - Muestra los datos personales del usuario afiliado.
 - Permite la actualización de algunos de estos datos.
5. **Agendar Cita**
 - Permite a los usuarios seleccionar una sucursal, un tipo de especialidad y un médico para agendar una cita.
 - Proporciona fechas y horas disponibles para la cita seleccionada.
6. **Crear Opinión**
 - Permite a los usuarios escribir y guardar opiniones sobre los médicos con los que han tenido citas.
7. **Historial de Citas**
 - Muestra una tabla con todas las citas pasadas y futuras del usuario.
 - Incluye detalles como el nombre del médico, la fecha, la hora y el consultorio de la cita.

Pantallas de Médico

1. **Perfil**
 - Muestra los datos personales del médico.
 - Permite la actualización de la foto de perfil del médico.
2. **Realizar Informe**
 - Permite al médico registrar un informe médico para un paciente específico en una fecha y hora seleccionada.

- Incluye campos para detalles como altura, peso, temperatura, alergias, diagnóstico y tratamiento.
3. **Calendario**
- Muestra un calendario interactivo donde el médico puede seleccionar una fecha y ver las citas programadas para ese día.
 - Permite ver los detalles del paciente para cada cita.

Vistas sin Iniciar Sesión

1. **Inicio:** Muestra la misión y visión del ISSSTE. Proporciona información básica y accesos rápidos a otras secciones de la aplicación.

Inicio

**ISSSTE**
INSTITUTO DE SEGURIDAD
Y SERVICIOS SOCIALES DE LOS
TRABAJADORES DEL ESTADO

Iniciar Sesión

Registrarse

InicioEquipo MédicoPreguntas Frecuentes

MISIÓN



Contribuir a satisfacer niveles de bienestar integral de los trabajadores al servicio del Estado, pensionados, jubilados y familiares derechohabientes, con el otorgamiento eficaz y eficiente de los seguros, prestaciones y servicios, con atención esmerada, respeto, calidad y cumplimiento siempre con los valores institucionales de honestidad, legalidad y transparencia.

VISIÓN



Posicionar al ISSSTE como un organismo público descentralizado que garantice la protección integral de los trabajadores de la Administración Pública Federal, pensionados, jubilados y sus familias de acuerdo con el perfil demográfico de la derechohabencia, con el otorgamiento de seguros, prestaciones y servicios de conformidad con la normatividad vigente, bajo códigos normados de calidad y calidez, con solvencia financiera, que permitan generar valores y prácticas que fomenten la mejora sostenida de bienestar, calidad de vida y el desarrollo del capital humano.

Dudas y contacto: atencionciudadana@issste.gob.mx o al 55-4000-1000

2. Equipo Médico

- **Explicación:** Muestra información sobre los médicos disponibles y las opiniones de los usuarios sobre ellos.
- **Altas:** Se pueden añadir opiniones sobre médicos.
- **Validaciones:** Se verifica que el médico seleccionado existe y que el comentario no está vacío.

Equipo Médico

ISSSTE
INSTITUTO DE SEGURIDAD
Y SERVICIOS SOCIALES DE LOS
TRABAJADORES DEL ESTADO

Iniciar Sesión Registrarse

Inicio Equipo Médico Preguntas Frecuentes

Opiniones equipo médico



Especialidad

Médico

Comentarios
Buen trato en general

Dudas y contacto: atencionciudadana@issste.gob.mx o al 55-4000-1000

- **Código:**

Metodo para rellenar los comentarios del medico:

```
private void cargarOpinionesDeMedico(Medico medico) {  
    DefaultTableModel opinionesTableModel = new DefaultTableModel();  
    opinionesTableModel.addColumn("Comentarios");  
    comentarios.setModel(opinionesTableModel);  
    if (medico != null) {  
        List<String> opiniones = getOpinionesByMedico(medico.getId());
```

```

        for (String opinion : opiniones) {

            opinionesTableModel.addRow(new Object[] {opinion});

        }

        System.out.println(medico.getCorreo());

        String img = cargarImg(Registrar_Informe.getIdMedico2(conn,
medico.getCorreo()));

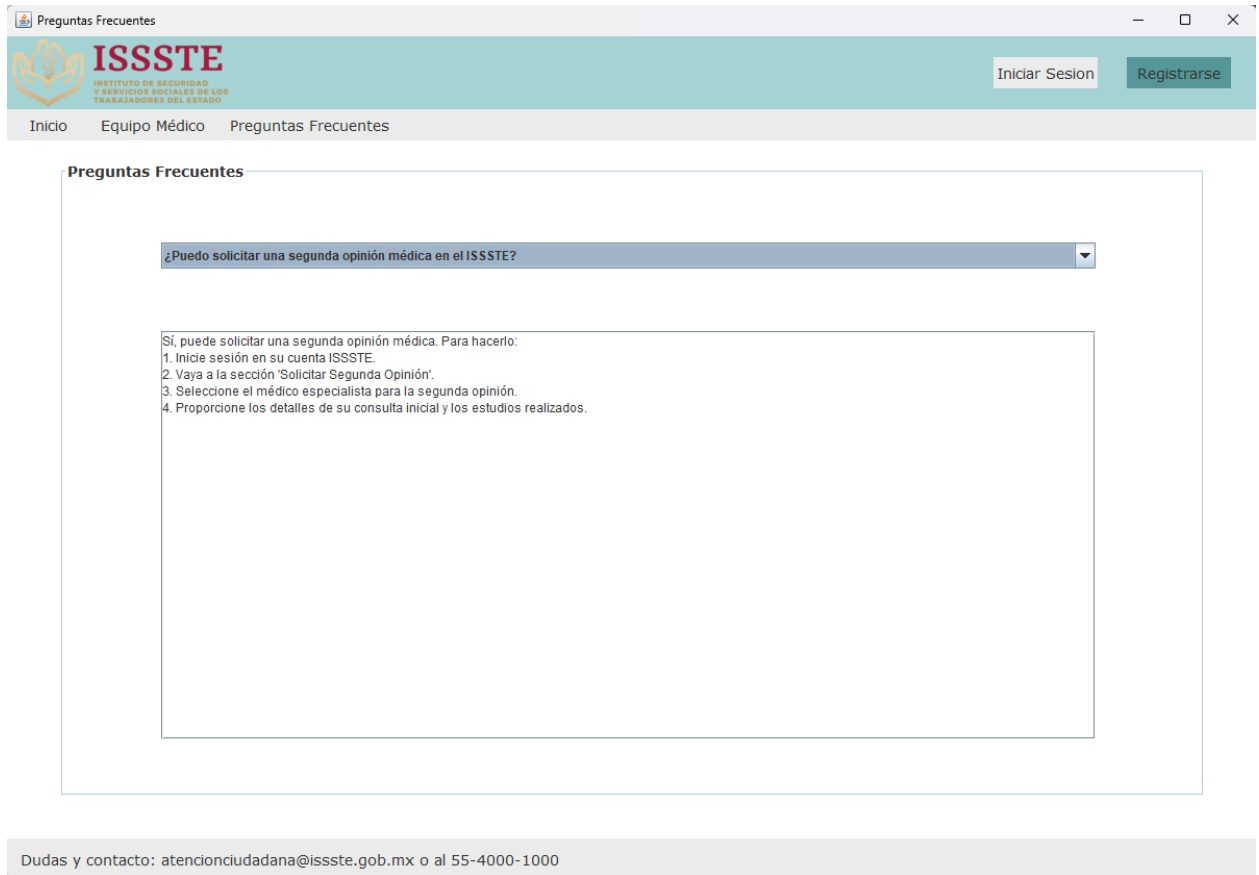
        if(img!=null){
            Image foto = getToolkit().getImage(img);
            fotoImg.setIcon(new ImageIcon(foto));
            //System.out.print(img);
        }else{
            fotoImg.setIcon(null);
        }
    }
}

private String cargarImg(){
    // Construir la sentencia SQL para obtener el nombre del usuario
    String nombre="";
    String sqlUsuario = "SELECT img FROM usuarios WHERE id = " +
user.getId();
    // Crear la declaración y ejecutar la consulta
    try (Statement stmtUsuario = conn.createStatement());
        ResultSet rs = stmtUsuario.executeQuery(sqlUsuario)) {

        if (rs.next()) {
            nombre = rs.getString("img");
        }
    } catch (SQLException e) {
        e.printStackTrace(); // Manejo adecuado de la excepción en tu aplicación
    }
    //System.out.println(nombre);
    return nombre;
}

```

3. **Preguntas Frecuentes:** Una sección donde los usuarios pueden seleccionar preguntas frecuentes y ver las respuestas correspondientes. Esta vista utiliza un `HashMap` para almacenar preguntas y respuestas, las cuales son mostradas en un `JComboBox` y un `TextArea`.



Código para llenar el combobox y el textArea

```
private void configurarComboBox() {  
    for (String pregunta : preguntasYRespuestas.keySet()) {  
        preguntaComboBox.addItem(pregunta);  
    }  
}  
  
private void mostrarRespuesta() {  
    String preguntaSeleccionada = (String) preguntaComboBox.getSelectedItem();  
    if (preguntaSeleccionada != null) {  
        String respuesta = preguntasYRespuestas.get(preguntaSeleccionada);
```

```

        respuestaTextArea.setText(respuesta);
    } else {
        respuestaTextArea.setText("");
    }
}

```

4. Iniciar Sesión

- **Explicación:** Formulario para que los usuarios inicien sesión ingresando su correo electrónico y contraseña.
- **Validaciones:** Verifica que el correo y la contraseña coinciden con un registro existente.

Registrate'. The footer contains contact information: 'Dudas y contacto: atencionciudadana@issste.gob.mx o al 55-4000-1000'."/>

Código para iniciar sesión:

```

public static Usuario iniciarSesion(Connection conn, String correo, String
contraseña){

```

```

// Construir la sentencia SQL para obtener las entidades

String sqlUsuario = "SELECT * FROM usuarios where correo = '"+correo+"' and
contraseña = '"+contraseña+"'";

// Crear la declaración y ejecutar la consulta
try (Statement stmtUsuario = conn.createStatement();
    ResultSet rs = stmtUsuario.executeQuery(sqlUsuario)) {

    if (rs.next()) {
        int id = rs.getInt("id");
        String nombre = rs.getString("nombre");
        String aP = rs.getString("apellido_p");
        String aM = rs.getString("apellido_m");
        String rfc = rs.getString("rfc");
        String curp = rs.getString("curp");
        String tipo = rs.getString("tipo");

        Usuario usuario = new
Usuario(rfc,curp,correo,contraseña,nombre,aM,aP);
        usuario.setTipo(tipo);
        usuario.setId(id);
        System.out.println(usuario.getCurp());
        return usuario;
    }
} catch (SQLException e) {
    e.printStackTrace(); // Manejo adecuado de la excepción en tu aplicación
}

return null;
}

```

5. Registro de Usuario

- **Explicación:** Permite a los nuevos usuarios registrarse proporcionando información personal.
- **Altas:** Registro de nuevos usuarios en la base de datos.
- **Validaciones:** Verifica que la CURP, RFC y correo no están registrados previamente.

The screenshot shows a web browser window titled "Registro Usuario". The header features the ISSSTE logo and the text "INSTITUTO DE SEGURIDAD Y SERVICIOS SOCIALES DE LOS TRABAJADORES DEL ESTADO". Below the header is a navigation bar with links: "Inicio", "Equipo Médico", and "Preguntas Frecuentes". The main content area is titled "Registro de usuario" and contains a registration form with the following fields: "Nombre", "Apellido Paterno", "Apellido Materno", "CURP", "RFC", "Correo Electronico", "Contraseña", and "Confirmar contraseña". Each field has a corresponding text input box. Below the fields is a green "Registrarse" button. At the bottom of the form, there is a link: "¿Ya tienes una cuenta?, [Inicia Sesión](#)". The footer of the page contains contact information: "Dudas y contacto: atencionciudadana@issste.gob.mx o al 55-4000-1000".

Codigo para registro de usuario:

```
public static boolean registrar(Connection conn, Usuario usuario) throws
SQLException{
    try{
        String sql1 = "SELECT * FROM usuarios WHERE curp = '"+
usuario.getCurp()+"'";
        String sql2 = "SELECT * FROM usuarios WHERE rfc = '" +
usuario.getRfc()+"'";
        String sql3 = "SELECT * FROM usuarios WHERE correo = '" +
usuario.getCorreo()+"'";
        Statement st = conn.createStatement();
```



```

Statement st1 = conn.createStatement();
Statement st2 = conn.createStatement();
ResultSet rs1 = st.executeQuery(sql1);
ResultSet rs2 = st1.executeQuery(sql2);
ResultSet rs3 = st2.executeQuery(sql3);
if (rs1.next()){
    JOptionPane.showMessageDialog(null, "Problemas al realizar el registro, la
curp ya ha sido registrada en otro usuario", "Advertencia",
JOptionPane.WARNING_MESSAGE);
    return false;
} else if(rs2.next()){
    JOptionPane.showMessageDialog(null, "Problemas al realizar el registro, el rfc
ya ha sido registrado en otro usuario", "Advertencia",
JOptionPane.WARNING_MESSAGE);
    return false;
} else if(rs3.next()){
    JOptionPane.showMessageDialog(null, "Problemas al realizar el registro, el
correo ya ha sido registrado en otro usuario", "Advertencia",
JOptionPane.WARNING_MESSAGE);
    return false;
} else {
    String sql4 = "INSERT INTO Usuarios (rfc, curp, correo, contraseña, nombre,
apellido_p, apellido_m, tipo) VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
    // Preparar la sentencia
    try (PreparedStatement stmt = conn.prepareStatement(sql4)) {
        // Establecer los parámetros de la sentencia con los valores del objeto
Usuario
        stmt.setString(1, usuario.getRfc());
        stmt.setString(2, usuario.getCurp());
        stmt.setString(3, usuario.getCorreo());
        stmt.setString(4, usuario.getContraseña());
        stmt.setString(5, usuario.getNombre());
        stmt.setString(6, usuario.getApellidoPaterno());
        stmt.setString(7, usuario.getApellidoMaterno());
        stmt.setString(8, "usuario");

        // Ejecutar la sentencia
        int filasInsertadas = stmt.executeUpdate();
        if (filasInsertadas > 0) {
            JOptionPane.showMessageDialog(null, "Registro realizado
correctamente");
            return true;

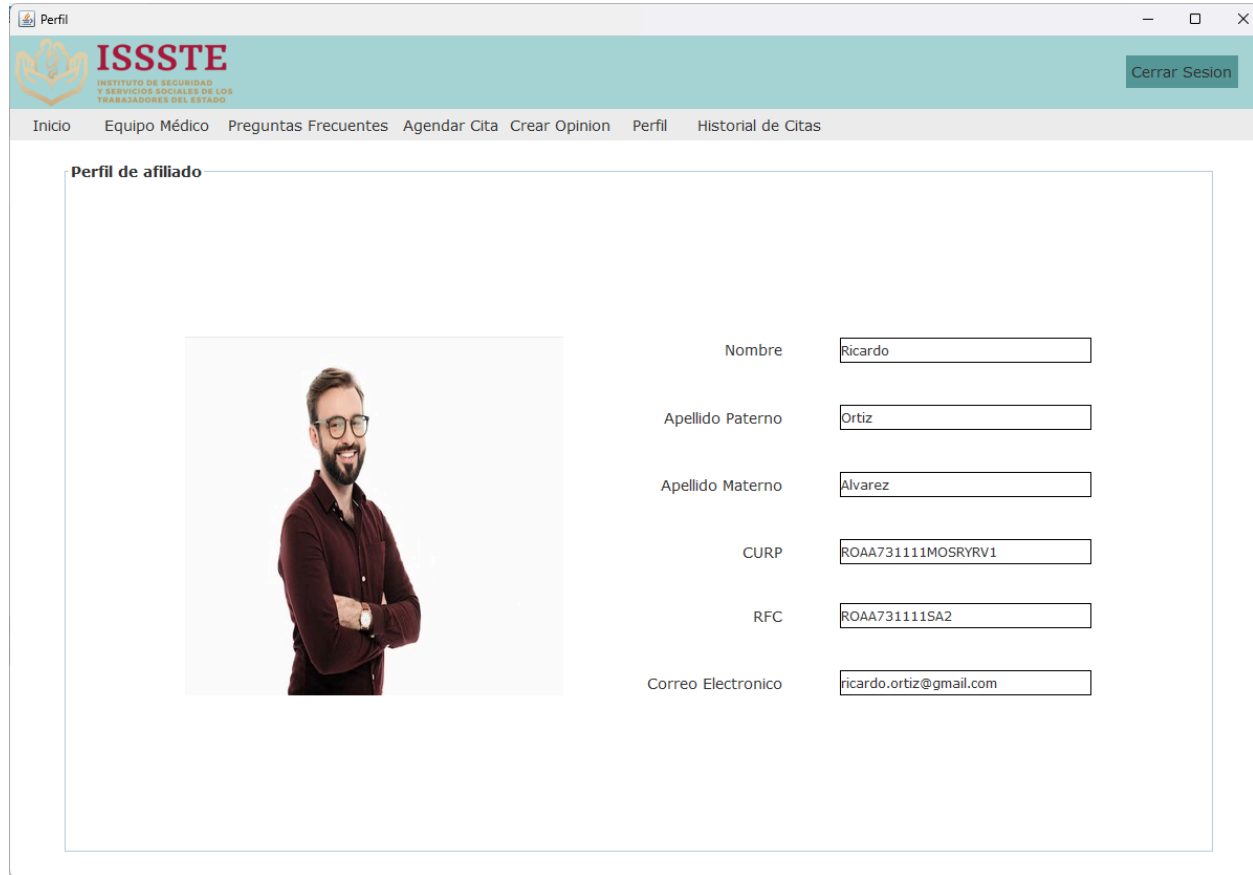
```

```
        }  
    }  
}  
} catch(SQLException e){  
    JOptionPane.showMessageDialog(null, "Problemas al realizar el registro",  
"Advertencia", JOptionPane.WARNING_MESSAGE);  
    return false;  
}  
return true;  
}
```

Vistas al Tener una Sesión

1. Perfil de Usuario

- **Explicación:** Muestra y permite actualizar la información personal del usuario.
- **Cambios:** Los usuarios pueden actualizar su información personal.



Perfil

ISSSTE
INSTITUTO DE SEGURIDAD
Y SERVICIOS SOCIALES DE LOS
TRABAJADORES DEL ESTADO

Cerrar Sesión

Inicio Equipo Médico Preguntas Frecuentes Agendar Cita Crear Opinion Perfil Historial de Citas

Perfil de afiliado

Nombre

Apellido Paterno

Apellido Materno

CURP

RFC

Correo Electronico

- **Código que permite rellenar los datos del usuario:**

```
public Inicio_Usuario(Usuario user, Connection conn) {  
    initComponents();  
    this.conn = conn;  
    this.user = user;  
    nombre.setText(user.getNombre());  
    apellido_m.setText(user.getApellidoMaterno());  
    apellido_p.setText(user.getApellidoPaterno());  
    correo.setText(user.getCorreo());  
    curp.setText(user.getCurp());  
    rfc.setText(user.getRfc());  
    img = cargarImg();  
}
```

```

        if(img!=null){
            Image foto = getToolkit().getImage(img);
            fotoImg.setIcon(new ImageIcon(foto));
            System.out.print(img);}
        // asignarIcono();
    }

```

Codigo que permite al usuario subir una fotografía de su rostro

```

private String cargarImg(){
    // Construir la sentencia SQL para obtener el nombre del usuario
    String nombre="";
    String sqlUsuario = "SELECT img FROM usuarios WHERE id = " +
user.getId();
    // Crear la declaración y ejecutar la consulta
    try (Statement stmtUsuario = conn.createStatement());
        ResultSet rs = stmtUsuario.executeQuery(sqlUsuario)) {

        if (rs.next()) {
            nombre = rs.getString("img");
        }
    } catch (SQLException e) {
        e.printStackTrace(); // Manejo adecuado de la excepción en tu aplicación
    }
    //System.out.println(nombre);
    return nombre;
}

```

```

private void insertarImg(String img1) throws SQLException,
FileNotFoundException{
    String insert = "UPDATE USUARIOS SET IMG = ? WHERE ID = ?";
    PreparedStatement pst = conn.prepareStatement(insert);
    pst.setString(1, img1);
    pst.setInt(2, user.getId());
    int i = pst.executeUpdate();
    if(i>0){
        JOptionPane.showMessageDialog(null,"Se ha guardado con exito");

    }else {
        JOptionPane.showMessageDialog(null,"Ha sucedido un problema al cargar
la imagen");    }    }

```

2. Agendar Cita

- **Explicación:** Permite a los usuarios agendar citas médicas seleccionando la sucursal, tipo de médico, fecha y hora.
- **Altas:** Inserción de nuevas citas en la base de datos.
- **Validaciones:** Verifica la disponibilidad de fechas y horas.

Agendar Cita

ISSSTE
INSTITUTO DE SEGURIDAD
Y SERVICIOS SOCIALES DE LOS
TRABAJADORES DEL ESTADO

Cerrar Sesión

Inicio Equipo Médico Preguntas Frecuentes Agendar Cita Crear Opinión Perfil Historial de Citas

Agendar Cita

Sucursal Oaxaca ▼

Tipo Dentista ▼
General
Dentista

Médico Dr. Roberto Sánchez ▼

Fecha 2024-06-03 ▼

Hora 08:30:00 ▼

Aceptar

Dudas y contacto: atencionciudadana@issste.gob.mx o al 55-4000-1000

- **Código**

Método para cargar el combo de sucursales:

// Método para obtener los datos de las sucursales desde la base de datos

```
public static void obtenerDatosSucursales(Connection conn,
JComboBox<String> sucursalComboBox) {
    try {
        String query = "SELECT id, entidad FROM Sucursales";
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(query);
        while (rs.next()) {
            int id = rs.getInt("id");
            String entidad = rs.getString("entidad");
            idSucursales.add(id);
        }
    }
}
```

```

        sucursalComboBox.addItem(entidad);
    }
    stmt.close();
} catch (SQLException e) {
    e.printStackTrace();
}
}
}

```

Metodo para cargar el combo de Medicos:

```

public static void obtenerDatosMedicos(Connection
conn,JComboBox<String> medicoComboBox, int Sucursal, String
tipo) {
    try {
        String query = "SELECT distinct nombre FROM Especialistas  
WHERE especialidad = '"+tipo+"'";
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(query);
        while (rs.next()) {
            String nombre = rs.getString("nombre");
            medicoComboBox.addItem(nombre);
        }
        stmt.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

Metodo para cargar combo de fechas:

```

public static void obtenerFechasDisponibles(Connection conn,
JComboBox<String> fechaComboBox, int idMedico) {
    try {
        System.out.println(idMedico);
        String query = "SELECT DISTINCT fecha FROM Citas WHERE  
id_medico = ? AND disponible = TRUE";
        PreparedStatement stmt = conn.prepareStatement(query);
        stmt.setInt(1, idMedico);
        ResultSet rs = stmt.executeQuery();
        while (rs.next()) {
            System.out.println("con");
            Date fecha = rs.getDate("fecha");
            fechaComboBox.addItem(fecha.toString());
        }
    }
}

```

```

    }
    stmt.close();
} catch (SQLException e) {
    e.printStackTrace();
}
}
}

```

Método para obtener las horas disponibles desde la base de datos

```

public static void obtenerHorasDisponibles(Connection conn,
JComboBox<String> horaComboBox, String fecha, int idMedico) {
    try {
        String query = "SELECT DISTINCT hora FROM Citas WHERE"
        fecha = ? AND id_medico = ? AND disponible = TRUE";
        PreparedStatement stmt = conn.prepareStatement(query);
        stmt.setString(1, fecha);
        stmt.setInt(2, idMedico);
        ResultSet rs = stmt.executeQuery();

        while (rs.next()) {
            Time hora = rs.getTime("hora");
            horaComboBox.addItem(hora.toString());
        }

        stmt.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

Metodo principal para cambiar el estado de la cita

```

public static void cambiarEstadoCita(Connection conn, int idCita, int
idAfiliado, String nombreAfiliado, String fechaAgendada, String

```

```

horaAgendada, String doctorAsignado, String fechaCita, String
horaCita, String consultorio, String tipo,int idMedico) {
    try {
        boolean captura;
        captura =
ControladorVista_Historial_De_Citas.ultimaCitaYaPaso(conn,
tipo,idAfiliado+""");
        if(captura){
            String query = "UPDATE Citas SET disponible = FALSE,
id_paciente = ? WHERE id = ?";
            PreparedStatement pstmt = conn.prepareStatement(query);
            pstmt.setInt(1, idAfiliado);
            pstmt.setInt(2, idCita);
            pstmt.executeUpdate();
            pstmt.close();
            System.out.println(tipo);
            // Ahora insertamos los datos en la tabla citas_afiliado
            String insertQuery = "INSERT INTO citas_afiliado (id_cita,
id_paciente, nombre_paciente, fecha_agendada, hora_agendada,
doctor_asignado, fecha_cita, hora_cita, consultorio, tipo,id_medico)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
            PreparedStatement insertStmt = conn.prepareStatement(insertQuery);
            insertStmt.setInt(1, idCita);
            insertStmt.setInt(2, idAfiliado);
            insertStmt.setString(3, nombreAfiliado);
            insertStmt.setString(4, fechaAgendada);
            insertStmt.setString(5, horaAgendada);
            insertStmt.setString(6, doctorAsignado);
            insertStmt.setString(7, fechaCita);
            insertStmt.setString(8, horaCita);
            insertStmt.setString(9, consultorio);
            insertStmt.setString(10, tipo);
            insertStmt.setInt(11, idMedico);
            insertStmt.executeUpdate();
            insertStmt.close();

            JOptionPane.showMessageDialog(null, "La cita ha sido agendada
correctamente");}
            else{
                JOptionPane.showMessageDialog(null, "Tienes una cita agendada
aún, intentalo de nuevo despues de finalizar tu cita");
            }
        } catch (SQLException e) {

```



```
        e.printStackTrace();  
    }  
}
```

Crear Opinión

- **Explicación:** Permite a los usuarios crear opiniones sobre los médicos después de una cita.
- **Altas:** Inserción de nuevas opiniones en la base de datos.
- **Validaciones:** Verifica que los campos no estén vacíos.

The screenshot shows a web browser window titled 'Crear_Opinion'. The header features the ISSSTE logo and name, with a 'Cerrar Sesión' button on the right. A navigation bar includes links: Inicio, Equipo Médico, Preguntas Frecuentes, Agendar Cita, Crear Opinión, Perfil, and Historial de Citas. The main form area is titled 'Crear Opinion' and contains two dropdown menus: 'Cargo' (set to 'General') and 'Nombre Doctor' (set to 'Dr. Juan Pérez (General)'). Below these is a text area labeled 'Opinion' containing the text 'Excelente servicio de medico, buena atencion, amable y atento'. An 'Enviar' button is at the bottom of the form. The footer provides contact information: 'Dudas y contacto: atencionciudadana@issste.gob.mx o al 55-4000-1000'.

Metodo para insertar la opinión

```
public boolean saveOpinion(int medicoId, String opinion) {  
    String query = "INSERT INTO opinionesequipomedico (id_medico, opinion)  
VALUES (?, ?)";
```

```
    try (PreparedStatement stmt = conn.prepareStatement(query)) {  
        stmt.setInt(1, medicoId);  
        stmt.setString(2, opinion);  
        int rowsAffected = stmt.executeUpdate();  
  
        return rowsAffected > 0;  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

```
        return false;
    }
}
```

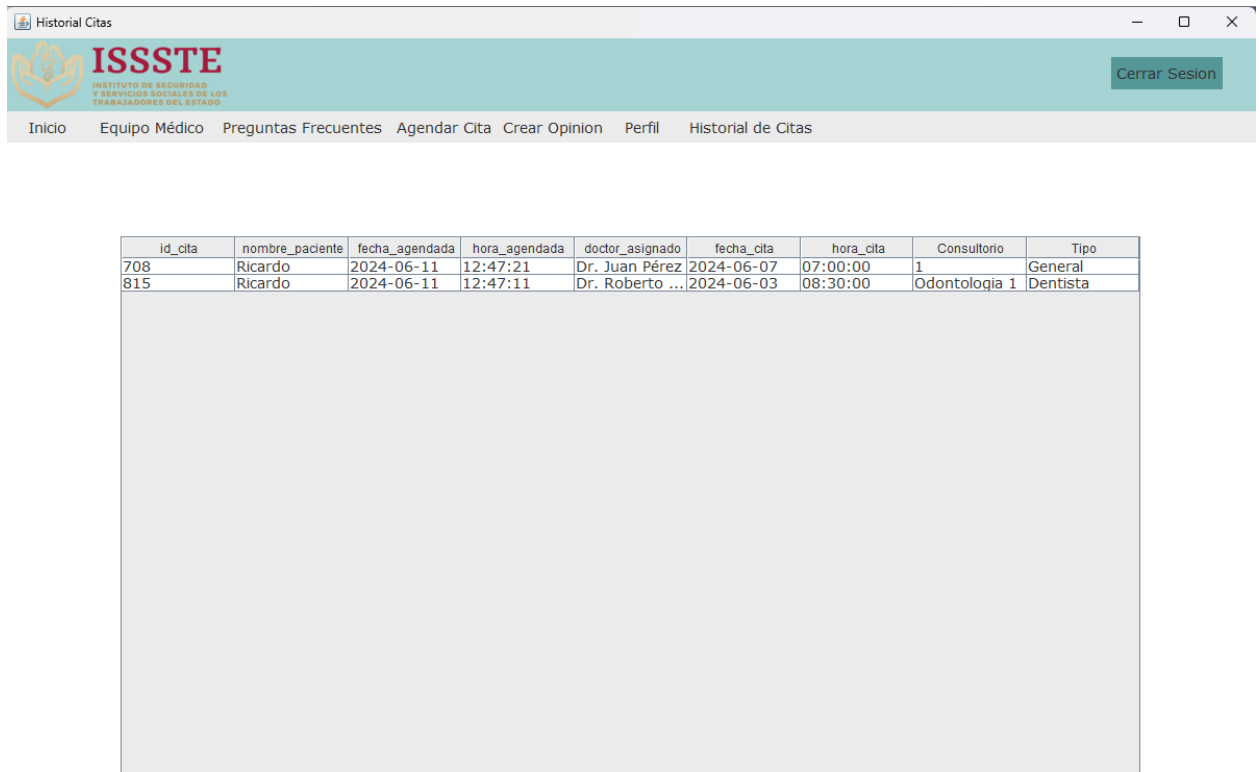
Metodo para guardar la opinión

```
private void guardarOpinion() {
    Medico medico = (Medico) Doctor.getSelectedItemAt();
    String opinion = opinionTextArea.getText();

    if (medico != null && !opinion.isEmpty()) {
        boolean success = saveOpinion(medico.getId(), opinion);
        if (success) {
            JOptionPane.showMessageDialog(this, "Opinión guardada exitosamente.");
        } else {
            JOptionPane.showMessageDialog(this, "Error al guardar la opinión.");
        }
    } else {
        JOptionPane.showMessageDialog(this, "Por favor, complete todos los campos.");
    }
}
```

6. Historial de Citas

- **Explicación:** Muestra un historial de las citas agendadas por el usuario.



Historial Citas

ISSSTE INSTITUTO DE SEGURIDAD Y SERVICIOS SOCIALES DE LOS TRABAJADORES DEL ESTADO

Cerrar Sesión

Inicio Equipo Médico Preguntas Frecuentes Agendar Cita Crear Opinion Perfil Historial de Citas

id_cita	nombre_paciente	fecha_agendada	hora_agendada	doctor_asignado	fecha_cita	hora_cita	Consultorio	Tipo
708	Ricardo	2024-06-11	12:47:21	Dr. Juan Pérez	2024-06-07	07:00:00	1	General
815	Ricardo	2024-06-11	12:47:11	Dr. Roberto ...	2024-06-03	08:30:00	Odontologia 1	Dentista

Metodo para mostrar la tabla

```
public static void mostrarDatos(Connection conn, JTable tabla,int id){
```

```
    DefaultTableModel modelo = new DefaultTableModel();
    modelo.addColumn("id_cita");
    modelo.addColumn("nombre_paciente");
    modelo.addColumn("fecha_agendada");
    modelo.addColumn("hora_agendada");
    modelo.addColumn("doctor_asignado");
    modelo.addColumn("fecha_cita");
    modelo.addColumn("hora_cita");
    modelo.addColumn("Consultorio");
    modelo.addColumn("Tipo");
    tabla.setModel(modelo);
    String sql="SELECT * FROM citas_afiliado WHERE id_paciente = "+id;
    String []datos = new String [9];
```

```

try{
    Statement st=conn.createStatement();
    ResultSet rs= st.executeQuery(sql);
    while(rs.next()){
        datos[0]=rs.getString(1);
        datos[1]=rs.getString(3);
        datos[2]=rs.getString(4);
        datos[3]=rs.getString(5);
        datos[4]=rs.getString(6);
        datos[5]=rs.getString(7);
        datos[6]=rs.getString(8);
        datos[7]=rs.getString(9);
        datos[8]=rs.getString(10);
        modelo.addRow(datos);
    }
    tabla.setModel(modelo);

} catch (SQLException ex) {

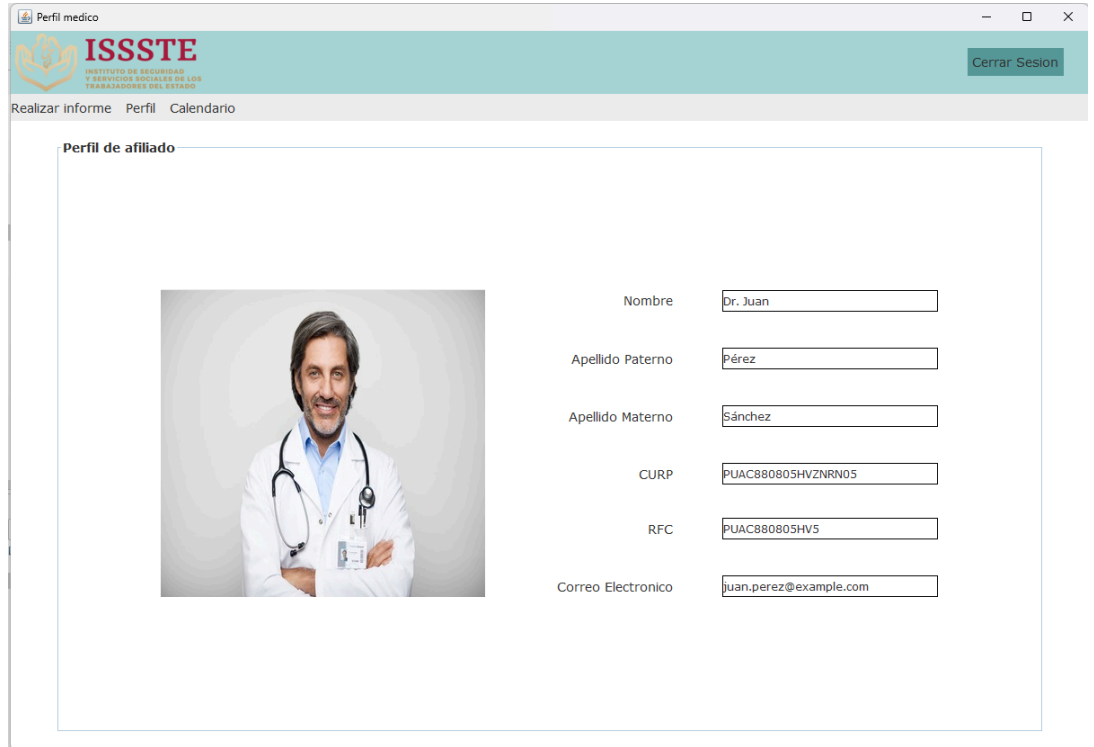
    Logger.getLogger(ControladorVista_Historial_De_Citas.class.getName()).log(Level.SEVERE, null, ex);
}
}

```

PERFIL DE MÉDICO

1. Perfil de Médico

- **Explicación:** Muestra y permite actualizar la información personal del médico.
- **Cambios:** Los médicos pueden actualizar su información personal y cargar una imagen.



- **Codigo que permite rellenar los datos del usuario:**

```
public Inicio_Usuario(Usuario user, Connection conn) {  
    initComponents();  
    this.conn = conn;  
    this.user = user;  
    nombre.setText(user.getNombre());  
    apellido_m.setText(user.getApellidoMaterno());  
    apellido_p.setText(user.getApellidoPaterno());  
    correo.setText(user.getCorreo());  
    curp.setText(user.getCurp());  
    rfc.setText(user.getRfc());  
    img = cargarImg();  
    if(img!=null){  
        Image foto = getToolkit().getImage(img);  
        fotoImg.setIcon(new ImageIcon(foto));  
    }  
}
```

```

        System.out.print(img);}
    // asignarIcono();
}

```

Codigo que permite al usuario subir una fotografía de su rostro

```

private String cargarImg(){
    // Construir la sentencia SQL para obtener el nombre del usuario
    String nombre="";
    String sqlUsuario = "SELECT img FROM usuarios WHERE id = " +
user.getId();
    // Crear la declaración y ejecutar la consulta
    try (Statement stmtUsuario = conn.createStatement();
        ResultSet rs = stmtUsuario.executeQuery(sqlUsuario)) {

        if (rs.next()) {
            nombre = rs.getString("img");
        }
    } catch (SQLException e) {
        e.printStackTrace(); // Manejo adecuado de la excepción en tu aplicación
    }
    //System.out.println(nombre);
    return nombre;
}

```

```

private void insertarImg(String img1) throws SQLException,
FileNotFoundException{
    String insert = "UPDATE USUARIOS SET IMG = ? WHERE ID = ?";
    PreparedStatement pst = conn.prepareStatement(insert);
    pst.setString(1, img1);
    pst.setInt(2, user.getId());
    int i = pst.executeUpdate();
    if(i>0){
        JOptionPane.showMessageDialog(null,"Se ha guardado con exito");

    }else {
        JOptionPane.showMessageDialog(null,"Ha sucedido un problema al cargar
la imagen");    }    }

```

Registrar Informe Médico

- **Explicación:** Permite a los médicos registrar informes médicos para los pacientes.
- **Altas:** Inserción de nuevos informes médicos en la base de datos.
- **Validaciones:** Verifica que no exista un informe duplicado para el mismo paciente, médico y fecha.

The screenshot shows a web application window titled 'Realizar informe'. The header features the ISSSTE logo and name, along with a 'Cerrar Sesión' button. Below the header is a navigation bar with links to 'Realizar informe', 'Perfil', and 'Calendario'. The main content area is titled 'Informe medico' and contains several input fields: 'Paciente' (a dropdown menu showing 'Fernanda Estrada Arroyo'), 'Fecha' (a date picker showing '2024-06-03'), 'Edad', 'Altura', 'Peso', 'Temperatura', 'Alergias', and 'Diagnostico'. At the bottom, there are two large text areas for 'Motivo' and 'Tratamiento', and a 'Registrar' button.

Metodo para registrar el informe médico:

```
public static void insertarInformeMedico(Connection conn, int id_paciente, int id_medico, String fecha, String altura, String peso, String temperatura, String alergias, String diagnostico, String tratamiento, String nombre_paciente, String nombre_medico, String edad, String motivo) {
```

```
    String sqlConsulta = "SELECT COUNT(*) FROM InformesMedicos WHERE fecha = ? AND id_medico = ? AND id_paciente = ?";
```



```
String sqlInsercion = "INSERT INTO InformesMedicos (id_paciente, id_medico, fecha, altura, peso, temperatura, alergias, Diagnostico, tratamiento, nombre_paciente, nombre_medico, edad, motivo) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
```

```
try {
```

```
    // Verificar si ya existe un informe médico con la misma fecha, id de médico e id de paciente
```

```
    PreparedStatement consultaStatement = conn.prepareStatement(sqlConsulta);
```

```
    consultaStatement.setString(1, fecha);
```

```
    consultaStatement.setInt(2, id_medico);
```

```
    consultaStatement.setInt(3, id_paciente);
```

```
    ResultSet resultadoConsulta = consultaStatement.executeQuery();
```

```
    resultadoConsulta.next(); // Mover el cursor al primer resultado
```

```
    int cantidadInformes = resultadoConsulta.getInt(1);
```

```
    if (cantidadInformes > 0) {
```

```
        JOptionPane.showMessageDialog(null, "Ya existe un informe médico con la misma fecha, médico y paciente. No se puede realizar la inserción.");
```

```
        return; // Salir del método si ya existe un informe médico con los mismos datos
```

```
    }
```

```
    // Si no existe un informe médico con los mismos datos, realizar la inserción
```

```
    PreparedStatement insercionStatement = conn.prepareStatement(sqlInsercion);
```

```
    insercionStatement.setInt(1, id_paciente);
```

```
    insercionStatement.setInt(2, id_medico);
```

```
insercionStatement.setString(3, fecha);  
  
insercionStatement.setString(4, altura);  
  
insercionStatement.setString(5, peso);  
  
insercionStatement.setString(6, temperatura);  
  
insercionStatement.setString(7, alergias);  
  
insercionStatement.setString(8, diagnostico);  
  
insercionStatement.setString(9, tratamiento);  
  
insercionStatement.setString(10, nombre_paciente);  
  
insercionStatement.setString(11, nombre_medico);  
  
insercionStatement.setString(12, edad);  
  
insercionStatement.setString(13, motivo);  
  
  
insercionStatement.executeUpdate();
```

```
        JOptionPane.showMessageDialog(null, "Informe médico insertado  
correctamente.");
```

```
    } catch (SQLException e) {
```

```
        JOptionPane.showMessageDialog(null, "Error al insertar el informe médico: " +  
e.getMessage());
```

```
    }
```

```
}
```

Calendario del Médico

- **Explicación:** Permite a los médicos ver su calendario de citas y seleccionar una cita para ver detalles del paciente.

Calendario

ISSSTE
INSTITUTO DE SEGURIDAD
Y SERVICIOS SOCIALES DE LOS
TRABAJADORES DEL ESTADO

Cerrar Sesión

Realizar informe Perfil Calendario

Perfil de afiliado

junio 2024

	dom	lun	mar	mié	jue	vie	sáb
22							1
23	2	3	4	5	6	7	8
24	9	10	11	12	13	14	15
25	16	17	18	19	20	21	22
26	23	24	25	26	27	28	29
27	30						

Hora: 09:00:00

Nombre Paciente: Nancy

Apellido Paterno: Estrada

Apellido Materno: García

Método para rellenar las horas dependiendo de la fecha

```
private void horaActionPerformed(java.awt.event.ActionEvent evt) {  
  
    try {  
  
        String nombreP = Registrar_Informe.obtenerNombre(conn, Fecha,  
hora.getSelectedItem().toString(), idMedico);  
  
        nombre.setText(nombreP);  
  
        int idPaciente = Registrar_Informe.obtenerIdPaciente(conn, nombreP, Fecha);  
  
        apellido_p.setText(Registrar_Informe.obtenerApellidoPaciente(conn, idPaciente));  
    }  
}
```

```

        apellido_m.setText(Registrar_Informe.obtenerApellidoMaternoPaciente(conn,
idPaciente));

    } catch (SQLException ex) {

        Logger.getLogger(Calendario.class.getName()).log(Level.SEVERE, null, ex);

    }

}

private String obtenerFechaSeleccionada() {

    int day = calendario.getDayChooser().getDay();

    int month = calendario.getMonthChooser().getMonth() + 1;

    int year = calendario.getYearChooser().getYear();

    return year + "-" + (month < 10 ? "0" + month : month) + "-" + (day < 10 ? "0" + day :
day);

}

```

Código Principal para la Conexión con la Base de Datos

El código principal para la conexión con la base de datos se encuentra en una clase que gestiona las conexiones con MySQL utilizando JDBC.

```

public class DatabaseConnection {

    private static final String URL = "jdbc:mysql://localhost:3306/issste";

    private static final String USER = "root";

    private static final String PASSWORD = "2000";

    public static Connection getConnection() throws SQLException {

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            return DriverManager.getConnection(URL, USER, PASSWORD);

        }

    }

}

```

```

    } catch (ClassNotFoundException e) {

        throw new SQLException("Error loading database driver", e);

    }

}

}

```

Implementación de los Patrones en el proyecto, beneficios y funcionamiento:

1. Builder

En nuestro proyecto de gestion de citas el patrón builder funciona así:

1. El Cliente crea una instancia del CitaBuilder.
2. El Cliente llama a los métodos del CitaBuilder para configurar los atributos de la Cita y asi poder una cita personalizada.
3. Despues el Cliente llama al método build() para obtener el objeto Cita completamente construido.

En el sistema de gestion citas médicas, el patrón Builder seimplementará para crear objetos Cita de manera flexible. Esto esespecialmente útil porque una Cita tiene muchos atributos, algunos de los cuales pueden ser opcionales.

Clase Cita (No implementada originalmente en el proyecto)

```

public class Cita {
    private int id;
    private int idPaciente;
    private int idMedico;private String fecha;
    private String hora;
    private String consultorio;
    private String tipo;
    private boolean disponible;
    // Constructor privado para evitar instanciación directa
    private Cita() {}
    // Getters y Setters ...

```

Clase CitaBuilder (No implementada originalmente en el proyecto)

```

public class CitaBuilder {
    private Cita cita;

    public CitaBuilder() {
        this.cita = new Cita();
    }

```

```

    }

    public CitaBuilder setId(int id) {
        cita.setId(id);
        return this;
    }

    public CitaBuilder setIdPaciente(int idPaciente) {
        cita.setIdPaciente(idPaciente);
        return this;
    }

    public CitaBuilder setIdMedico(int idMedico) {
        cita.setIdMedico(idMedico);
        return this;
    }

    public CitaBuilder setFecha(String fecha) {
        cita.setFecha(fecha);
        return this;
    }

    public CitaBuilder setHora(String hora) {
        cita.setHora(hora);
        return this;
    }

    public CitaBuilder setConsultorio(String consultorio) {
        cita.setConsultorio(consultorio);
        return this;
    }

    public CitaBuilder setTipo(String tipo) {
        cita.setTipo(tipo);
        return this;
    }

    public CitaBuilder setDisponible(boolean disponible) {
        cita.setDisponible(disponible);
        return this;
    }

    public Cita build() {
        return cita;
    }
}

```

Cambios en el proyecto:

Clase agendarCita metodo AceptarMouseClicked

```

// TODO add your handling code here:\
    hora = Hora.getSelectedItem().toString();
    int idCita=Formato_Citas.obtenerIdCita(conn, idMedico, fecha, hora);
    LocalDateTime now = LocalDateTime.now();

```

```

        // Definir un formato personalizado para la fecha y hora
        DateTimeFormatter dateFormatter =
DateTimeFormatter.ofPattern("yyyy-MM-dd");
        DateTimeFormatter timeFormatter =
DateTimeFormatter.ofPattern("HH:mm:ss");

        // Formatear la fecha y la hora como cadenas de texto
        String fechaActual = now.format(dateFormatter);
        String horaActual = now.format(timeFormatter);
        Cita cita = new CitaBuilder()
        .setId(idCita)
        .setIdPaciente(user.getId())
        .setIdMedico(idMedico)
        .setFecha(fecha)
        .setHora(hora)
        .setConsultorio(consultorio)
        .setTipo(tipo1)
        .setDisponible(false)
        .build();
        Formato_Citas.cambiarEstadoCita(conn,
idCita,user.getId(),user.getNombre(),fechaActual,horaActual,nomDoc,fecha,hora,
consultorio,tipo1,idMedico,cita);
        Inicio_Usuario ho = new Inicio_Usuario(user,conn);
        this.setVisible(false);
        ho.setVisible(true);

```

Pruebas

Al registrar una cita debe aparecer en la consola este mensaje, dado que es un patron que trabaja silenciosamente solo es un mensaje para saber que se registro la cita correctamente.

```

run:
Cita{id=688, idPaciente=61, idMedico=11, fecha='2024-06-03', hora='07:00:00', consultorio='1', tipo='General', disponible=false}
BUILD SUCCESSFUL (total time: 0 seconds)

```

2. Abstract Factory

El patrón Abstract Factory funciona de la siguiente manera:

1. El Cliente (Main) utiliza una FabricaAbstracta para crear objetos.
2. La FabricaAbstracta define métodos para crear cada tipo de objeto (médico, paciente, cita).
3. Las fábricas específicas (FabricaMedicosGenerales, FabricaDentistas) implementan estos métodos para crear objetos específicos (médicos generales, dentistas, etc.).

En nuestro sistema de citas médicas, implementamos el patrón Abstract Factory para manejar dos tipos de médicos: médicos generales y dentistas. Cada tipo de médico tiene su propia familia de

objetos relacionados (pacientes y citas), **este trabaja en conjunto con el patron builder**, ya que realiza una pequeña simulación al momento de registrar una cita, primero entra el patrón builder haciendo el registro y creando un objeto cita para después entrar el patrón abstract factory donde dice el procedimiento en el que sucede el registro de la cita.

Código e implementación

1. Clases abstractas (Nuevas clases)

```
public interface FabricaAbstracta {
    Medico crearMedico();
    Paciente crearPaciente();
    Cita crearCita();
}
public interface Medico {
    void atenderPaciente();}
public interface Paciente {
    void solicitarCita();
}
public interface Cita {
    void programar();
}
```

2. Fabricas (Nuevas Clases)

```
public class FabricaMedicosGenerales implements FabricaAbstracta {
    @Override
    public Medico crearMedico() {
        return new MedicoGeneral();
    }
    @Override
    public Paciente crearPaciente() {
        return new PacienteGeneral();
    }
    @Override
    public Cita crearCita() {
        return new CitaGeneral();
    }
}
public class FabricaDentistas implements FabricaAbstracta {
    @Override
    public Medico crearMedico() {
        return new Dentista();
    }
    @Override
    public Paciente crearPaciente() {
        return new PacienteDental();
    }
    @Override
    public Cita crearCita() {
        return new CitaDentista();
    }
}
```


3. Productos (Nuevas clases)

```
public class MedicoGeneral implements Medico {
    @Override
    public void atenderPaciente() {
        System.out.println("Médico general atendiendo paciente...");
    }
}

public class Dentista implements Medico {
    @Override
    public void atenderPaciente() {
        System.out.println("Dentista atendiendo paciente...");
    }
}

public class PacienteGeneral implements Paciente {
    @Override
    public void solicitarCita() {
        System.out.println("Paciente general solicitando cita...");
    }
}

public class PacienteDental implements Paciente {
    @Override
    public void solicitarCita() {
        System.out.println("Paciente dental solicitando cita...");
    }
}

public class CitaGeneral implements Cita {
    @Override
    public void programar() {
        System.out.println("Cita general programada.");
    }
}

public class CitaDentista implements Cita {
    @Override
    public void programar() {
        System.out.println("Cita dental programada.");
    }
}
```

4. Cliente

```
FabricaAbstracta fabricaMedicosGenerales = new
FabricaMedicosGenerales();
Medico medicoGeneral =
fabricaMedicosGenerales.crearMedico();
Paciente pacienteGeneral =
fabricaMedicosGenerales.crearPaciente();
Cita citaGeneral = fabricaMedicosGenerales.crearCita();
medicoGeneral.atenderPaciente();
pacienteGeneral.solicitarCita();
citaGeneral.programar();
// Crear una fábrica de dentistas
```

```

FabricaAbstracta fabricaDentistas = new FabricaDentistas();
Medico dentista = fabricaDentistas.crearMedico();
Paciente pacienteDental = fabricaDentistas.crearPaciente();
Cita citaDentista = fabricaDentistas.crearCita();
dentista.atenderPaciente();
pacienteDental.solicitarCita();
citaDentista.programar();
}

```

Pruebas.

Al momento de registrar una cita debe aparecer la siguiente salida

```

run:
Medico general atendiendo paciente...
Paciente general solicitando cita...
Cita general programada.
Dentista atendiendo paciente...
Paciente dental solicitando cita...
Cita dental programada.
BUILD SUCCESSFUL (total time: 1 second)

```

3. Singleton

En nuestra implementación, singleton funciona de la siguiente manera son:

La clase Conexion se arrastrará a todas las clases, al iniciar sesión la conexión se moverá de pestaña en pestaña para evitar que un usuario tenga más de una conexión, además al cerrar sesión también se cerrará la instancia de la conexión para ese usuario.

Clase Conexión antigua:

```

public class Conexion {
    Connection conectar;
    String usuario = "root";
    String contrasenia = "2000";
    String bd = "ISSSTE_AGIL";
    String ip = "localhost";
    String puerto = "3306";
    String cadena = "jdbc:mysql://" + ip + ":" + puerto + "/" + bd;

    public Connection estableceConexion(){
        try{
            conectar = DriverManager.getConnection(cadena, usuario, contrasenia);
            JOptionPane.showMessageDialog(null, "Se conecto correctamente a la base de
datos");
        } catch (Exception e){
            JOptionPane.showMessageDialog(null, "Problemas en la
conexion" + e.toString());
        }
    }
}

```

```

    }
    return conectar;
}
}

```

Clase Conexión nueva:

```

public class ConexionBD {
    // Atributo estático que guarda la única instancia
    private static ConexionBD instancia;
    private Connection conexion;

    // Constructor privado para evitar instanciación externa
    private ConexionBD() {
        try {
            // Parámetros de conexión (ajustar según tu configuración)
            String url = "jdbc:mysql://localhost:3306/issste";
            String usuario = "tu_usuario";
            String contraseña = "tu_contraseña";

            // Crear la conexión
            this.conexion = DriverManager.getConnection(url, usuario, contraseña);
            System.out.println("Conexión establecida correctamente.");
        } catch (SQLException e) {
            System.err.println("Error al conectar a la BD: " + e.getMessage());
        }
    }

    // Método estático para obtener la instancia única
    public static ConexionBD getInstance() {
        if (instancia == null) {
            instancia = new ConexionBD();
        }
        return instancia;
    }

    // Método para obtener la conexión
    public Connection getConexion() {
        return conexion;
    }

    // Método para cerrar la conexión (opcional)
    public void cerrarConexion() {
        try {
            if (conexion != null && !conexion.isClosed()) {
                conexion.close();
                System.out.println("Conexión cerrada.");
            }
        } catch (SQLException e) {
            System.err.println("Error al cerrar la conexión: " + e.getMessage());
        }
    }
}

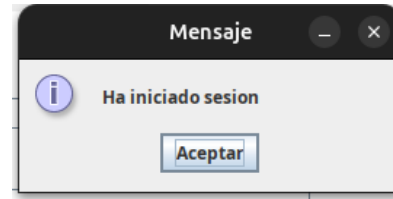
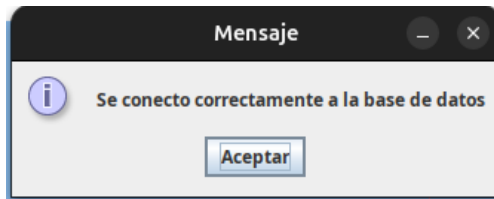
```

```

    }
}
}

```

Pruebas:



4. Facade

El patrón Facade funciona de la siguiente manera:

1. El Cliente (Main) interactúa con la Fachada (FacadeAutenticacion) para realizar operaciones de autenticación y registro.
2. La Fachada coordina las llamadas a los subsistemas (gestión de usuarios y validaciones) para realizar las operaciones necesarias.
3. El Cliente no necesita conocer los detalles de implementación de los subsistemas, lo que simplifica su código.

Clases Involucradas

1. FacadeAutenticacion: Proporciona métodos simplificados para iniciar sesión y registrar usuarios.
2. Usuario: Representa a un usuario en el sistema.
3. Main: Cliente que utiliza la Fachada para realizar operaciones de autenticación y registro.

Código:

Clase FacadeAutenticacion(Nueva):

En esta clase administramos el inicio y registro de sesión para no sobrecargar las vistas, así que se movió el código de esa clase dejando solo los accesos de los métodos en el registro e inicio de sesión del usuario.

```

public class FacadeAutenticacion {

    // Método para iniciar sesión
    public static Usuario iniciarSesion(Connection conn, String correo, String
contraseña) {
        String sqlUsuario = "SELECT * FROM usuarios WHERE correo = ? AND contraseña =
?";

        try (PreparedStatement stmtUsuario = conn.prepareStatement(sqlUsuario)) {
            stmtUsuario.setString(1, correo);
            stmtUsuario.setString(2, contraseña);

```

```

        try (ResultSet rs = stmtUsuario.executeQuery()) {
            if (rs.next()) {
                int id = rs.getInt("id");
                String nombre = rs.getString("nombre");
                String aP = rs.getString("apellido_p");
                String aM = rs.getString("apellido_m");
                String rfc = rs.getString("rfc");
                String curp = rs.getString("curp");
                String tipo = rs.getString("tipo");

                Usuario usuario = new Usuario(rfc, curp, correo, contraseña,
nombre, aM, aP);

                usuario.setTipo(tipo);
                usuario.setId(id);

                System.out.println("Usuario encontrado: " + usuario.getCurp());
                return usuario;
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }

    JOptionPane.showMessageDialog(null, "Correo o contraseña incorrectos",
"Error", JOptionPane.ERROR_MESSAGE);
    return null;
}

// Método para registrar un nuevo usuario
public static boolean registrar(Connection conn, Usuario usuario) {
    try {
        // Verificar si el CURP, RFC o correo ya están registrados
        if (existeRegistro(conn, "curp", usuario.getCurp())) {
            JOptionPane.showMessageDialog(null, "La CURP ya ha sido registrada en
otro usuario", "Advertencia", JOptionPane.WARNING_MESSAGE);
            return false;
        } else if (existeRegistro(conn, "rfc", usuario.getRfc())) {
            JOptionPane.showMessageDialog(null, "El RFC ya ha sido registrado en
otro usuario", "Advertencia", JOptionPane.WARNING_MESSAGE);
            return false;
        } else if (existeRegistro(conn, "correo", usuario.getCorreo())) {
            JOptionPane.showMessageDialog(null, "El correo ya ha sido registrado
en otro usuario", "Advertencia", JOptionPane.WARNING_MESSAGE);
            return false;
        } else {
            // Insertar el nuevo usuario
            String sql = "INSERT INTO usuarios (rfc, curp, correo, contraseña,
nombre, apellido_p, apellido_m, tipo) VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
            try (PreparedStatement stmt = conn.prepareStatement(sql)) {
                stmt.setString(1, usuario.getRfc());
                stmt.setString(2, usuario.getCurp());
                stmt.setString(3, usuario.getCorreo());

```

```

        stmt.setString(4, usuario.getContraseña());
        stmt.setString(5, usuario.getNombre());
        stmt.setString(6, usuario.getApellidoPaterno());
        stmt.setString(7, usuario.getApellidoMaterno());
        stmt.setString(8, "usuario"); // Tipo de usuario por defecto

        int filasInsertadas = stmt.executeUpdate();
        if (filasInsertadas > 0) {
            JOptionPane.showMessageDialog(null, "Registro realizado
correctamente", "Éxito", JOptionPane.INFORMATION_MESSAGE);
            return true;
        }
    }
} catch (SQLException e) {
    JOptionPane.showMessageDialog(null, "Problemas al realizar el registro",
"Error", JOptionPane.ERROR_MESSAGE);
    e.printStackTrace();
}

return false;
}

// Método auxiliar para verificar si un campo ya está registrado
private static boolean existeRegistro(Connection conn, String campo, String
valor) throws SQLException {
    String sql = "SELECT * FROM usuarios WHERE " + campo + " = ?";
    try (PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, valor);
        try (ResultSet rs = stmt.executeQuery()) {
            return rs.next(); // Retorna true si existe un registro con ese valor
        }
    }
}
}
}

```

Clase Usuario(Nueva)

```

public class Usuario {
    private String rfc, curp, correo, contraseña, nombre, apellidoMaterno,
apellidoPaterno, tipo;
    private int id;
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public void setTipo(String tipo) {

```

```

        this.tipo = tipo;
    }

    public Usuario(String rfc, String curp, String correo, String contraseña, String
nombre, String apellidoMaterno, String apellidoPaterno) {
        this.rfc = rfc;
        this.curp = curp;
        this.correo = correo;
        this.contraseña = contraseña;
        this.nombre = nombre;
        this.apellidoMaterno = apellidoMaterno;
        this.apellidoPaterno = apellidoPaterno;
    }
//Getters y Setters

```

Cliente Iniciar Sesión:

```

private void iniciar_SesionMouseClicked(java.awt.event.MouseEvent evt) {
    // TODO add your handling code here:
    String correoText = correo.getText();
    String contraseñaText = new String(contraseña.getPassword());
    if(correoText.isEmpty() || contraseñaText.isEmpty()){
        JOptionPane.showMessageDialog(this, "Rellena todos los campos", "Error",
JOptionPane.ERROR_MESSAGE);
        return;
    }else{
        Usuario usuario =
FacadeAutenticacion.iniciarSesion(conn,correoText,contraseñaText);
        if (usuario==null){
            JOptionPane.showMessageDialog(this, "El usuario y/o contraseña son
incorrectos", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }else if(usuario.getTipo().equals("usuario")){
            JOptionPane.showMessageDialog(null,"Ha iniciado sesion");
            this.setVisible(false);
            Inicio_Usuario inicio = new Inicio_Usuario(usuario,conn);
            inicio.setVisible(true);
        } else if(usuario.getTipo().equals("medico_general")){
            JOptionPane.showMessageDialog(null,"Ha iniciado sesion");
            this.setVisible(false);
            Vista_Doctor inicio = new Vista_Doctor(usuario,conn);
            inicio.setVisible(true);
        }
    }
}
}

```

Cliente RegistrarUsuario:

```

// TODO add your handling code here:
String rfcText = rfc.getText();
String curpText = curp.getText();
String Nombre = nombre.getText();
String correoText = correo.getText();

```

```

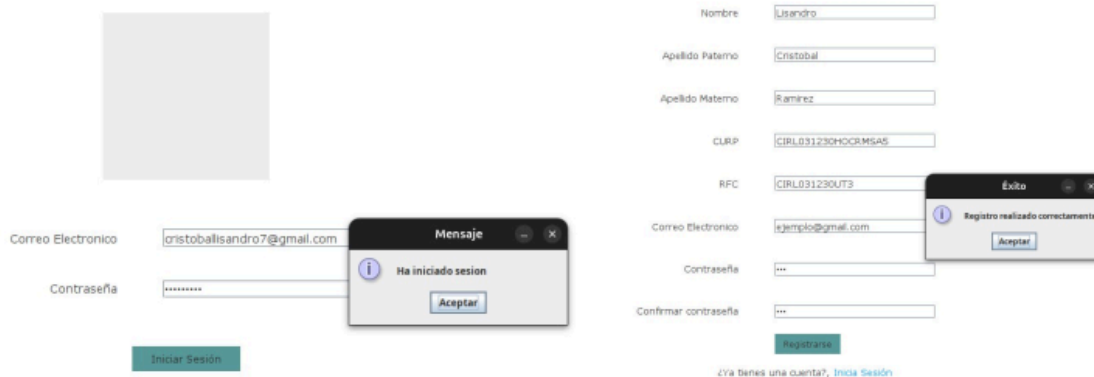
        String contraseñaText = new String(contraseña.getPassword());
        String confirmarContraseñaText = new
String(confirmar_contraseña.getPassword());
        String apellidoMaternoText = apellido_m.getText();
        String apellidoPaternoText = apellido_p.getText();
        if (Nombre.length()==0 || correoText.length()==0 ||contraseñaText.length()==0
|| confirmarContraseñaText.length()==0 ||apellidoMaternoText.length()==0
||apellidoPaternoText.length()==0 ){
            JOptionPane.showMessageDialog(this, "Rellena todos los campos",
"Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
        if(validarCurp(curpText) && validarRFC(rfcText)){

            if (!contraseñaText.equals(confirmarContraseñaText)) {
                JOptionPane.showMessageDialog(this, "Las contraseñas no coinciden",
"Error", JOptionPane.ERROR_MESSAGE);
                return;
            }

            // Crear el objeto Usuario si la contraseña coincide
            Usuario usuario = new Usuario(rfcText, curpText, correoText, contraseñaText,
Nombre, apellidoMaternoText, apellidoPaternoText);
            if(FacadeAutenticacion.registrar(conn, usuario)){
                Iniciar_Sesion iniciar = new Iniciar_Sesion(conn);
                iniciar.setVisible(true);
                this.setVisible(false);
            }
        }
    }
}

```

Pruebas:



The image displays two screenshots of a web application interface for user authentication.

Left Screenshot (Login): Shows a login form with fields for "Correo Electronico" (Email) and "Contraseña" (Password). The email field contains "crisoballisandro7@gmail.com". A "Mensaje" dialog box is overlaid, displaying "Ha iniciado sesion" (You have logged in) with an "Aceptar" (Accept) button. Below the form is a green "Iniciar Sesión" (Login) button.

Right Screenshot (Registration): Shows a registration form with fields for "Nombre" (Name), "Apellido Paterno" (Paternal Surname), "Apellido Materno" (Maternal Surname), "CURP", "RFC", "Correo Electronico" (Email), "Contraseña" (Password), and "Confirmar contraseña" (Confirm Password). The fields are filled with: "Luisandro", "Cristobal", "Ramirez", "CIRL031230H00RMISAS", "CIRL031230UT3", "ejemplo@gmail.com", and masked passwords. A green "Registrarse" (Register) button is at the bottom. An "Éxito" (Success) dialog box is overlaid, displaying "Registro realizado correctamente" (Registration completed successfully) with an "Aceptar" (Accept) button. Below the form is a link: "¿Ya tienes una cuenta?, Inicia Sesión" (Do you already have an account?, Login).

5. Composite

Composite es un patrón de diseño estructural que te permite componer objetos en estructuras de árbol y trabajar con esas estructuras como si fueran objetos individuales.

En este sentido, nuestro proyecto aplicando el patrón composite funciona con las siguientes clases:

- La clase *Usuario* a través de una interfaz común que declara un método para mostrar la información de los usuarios, la clase *Medico* también implementa este método.
- En este sentido, *Usuario* podría ser forma de una superclase a la que pertenezcan los usuarios, ya que sería un componente del hospital (*ComponenteHospital*) al igual que *Medico*.
- En este patrón también se implementará una clase llamada *GrupoUsuarios* para crear una agrupación de diferentes usuarios.
- Se creará la vista *Registro_Grupo_Usuarios* se registran los grupos de usuarios.

Este patrón trabaja junto con el patrón Prototype, ya que al registrar un nuevo usuario o grupo de usuarios mediante el patrón Composite, el patrón Prototype clona este usuario o grupo de usuarios.

Interfaz ComponenteHospital:

```
public interface ComponenteHospital {
    void mostrarInformacion();
    void agregar(ComponenteHospital componente); // Solo para composites
    void eliminar(ComponenteHospital componente); // Solo para composites
    List<ComponenteHospital> getSubordinados(); // Solo para composites
}
```

Clase Usuario:

```
public class Usuario implements Cloneable, ComponenteHospital{
    private String rfc, curp, correo, contraseña, nombre, apellidoMaterno,
    apellidoPaterno, tipo;
    private int id;
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public void setTipo(String tipo) {
        this.tipo = tipo;
    }

    public Usuario(String rfc, String curp, String correo, String contraseña, String
    nombre, String apellidoMaterno, String apellidoPaterno) {
        this.rfc = rfc;
        this.curp = curp;
        this.correo = correo;
    }
}
```

```
        this.contraseña = contraseña;
        this.nombre = nombre;
        this.apellidoMaterno = apellidoMaterno;
        this.apellidoPaterno = apellidoPaterno;
    }

    public String getTipo() {
        return tipo;
    }

    public String getRfc() {
        return rfc;
    }

    public String getCurp() {
        return curp;
    }

    public String getCorreo() {
        return correo;
    }

    public String getContraseña() {
        return contraseña;
    }

    public String getNombre() {
        return nombre;
    }

    public String getApellidoMaterno() {
        return apellidoMaterno;
    }

    public String getApellidoPaterno() {
        return apellidoPaterno;
    }

    public void setRfc(String rfc) {
        this.rfc = rfc;
    }

    public void setCurp(String curp) {
        this.curp = curp;
    }

    public void setCorreo(String correo) {
        this.correo = correo;
    }

    public void setContraseña(String contraseña) {
        this.contraseña = contraseña;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
```

```

    }

    public void setApellidoMaterno(String apellidoMaterno) {
        this.apellidoMaterno = apellidoMaterno;
    }

    public void setApellidoPaterno(String apellidoPaterno) {
        this.apellidoPaterno = apellidoPaterno;
    }

    // Implementación del patrón Prototype
    @Override
    public Usuario clone() {
        try {
            Usuario clon = (Usuario) super.clone();
            clon.setId(0); // Para que la base de datos asigne un nuevo ID

            // Modificar correo para evitar conflictos en la base de datos
            clon.setCorreo(this.correo.replace("@", "+clon@"));

            // Modificar RFC y CURP (opcional)
            clon.setRfc(this.rfc + "C");
            clon.setCarp(this.carp + "C");

            return clon;
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException("Error al clonar Usuario", e);
        }
    }

    //implementación del patrón composite
    @Override
    public void mostrarInformacion() {
        System.out.println("Usuario: " + nombre + " " + apellidoPaterno + " - RFC: "
+ rfc);
    }

    //implementación del patrón composite
    @Override
    public void agregar(ComponenteHospital componente) {
        throw new UnsupportedOperationException("No aplicable para usuarios
individuales");
    }

    //implementación del patrón composite
    @Override
    public void eliminar(ComponenteHospital componente) {
        throw new UnsupportedOperationException("No aplicable para usuarios
individuales");
    }

    //implementación del patrón composite
    @Override
    public List<ComponenteHospital> getSubordinados() {
        return Collections.emptyList(); // No tiene subordinados
    }

```

```
}
```

Clase GrupoUsuarios:

```
public class GrupoUsuarios {
    private List<Usuario> usuarios;
    private Connection conn; // Conexión a la base de datos

    // Constructor
    public GrupoUsuarios(Connection conn) {
        this.usuarios = new ArrayList<>();
        this.conn = conn;
    }

    // Agregar usuario al grupo
    public void agregarUsuario(Usuario usuario) {
        if (usuario != null) {
            usuarios.add(usuario);
        }
    }

    // Eliminar usuario del grupo
    public void eliminarUsuario(Usuario usuario) {
        usuarios.remove(usuario);
    }

    // Registrar todos los usuarios del grupo
    public boolean registrarUsuarios() {
        if (usuarios.isEmpty()) {
            return false;
        }

        try {
            for (Usuario usuario : usuarios) {
                RegistrarUsuario.registrar(conn, usuario);
            }
            usuarios.clear(); // Limpiar la lista después de registrar
            return true;
        } catch (SQLException ex) {
            ex.printStackTrace();
            return false;
        }
    }

    // Obtener la lista de usuarios
    public List<Usuario> getUsuarios() {
        return usuarios;
    }

    // Verificar si el grupo está vacío
    public boolean estaVacio() {
        return usuarios.isEmpty();
    }
}
```

Clase Medico:

```
public class Medico implements Cloneable, ComponenteHospital{
    private int id;
    private String nombre;
    private String especialidad;
    private String correo;
    public Medico(int id, String nombre, String especialidad, String correo) {
        this.id = id;
        this.nombre = nombre;
        this.especialidad = especialidad;
        this.correo=correo;
    }

    public int getId() {
        return id;
    }

    public String getCorreo() {
        return correo;
    }

    public void setCorreo(String correo) {
        this.correo = correo;
    }

    public String getNombre() {
        return nombre;
    }

    public String getEspecialidad() {
        return especialidad;
    }

    // Implementación del patrón Prototype
    @Override
    public Medico clone() {
        try {
            return (Medico) super.clone(); // Clonación superficial
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException("Error al clonar Medico", e);
        }
    }

    //implementación del patrón composite
    @Override
    public void mostrarInformacion() {
        System.out.println("Médico: " + nombre + " - Especialidad: " + especialidad);
    }

    //implementación del patrón composite
    @Override
    public void agregar(ComponenteHospital componente) {
        throw new UnsupportedOperationException("No aplicable para usuarios individuales");
    }
}
```

```

    }

    //implementación del patrón composite
    @Override
    public void eliminar(ComponenteHospital componente) {
        throw new UnsupportedOperationException("No aplicable para usuarios individuales");
    }

    //implementación del patrón composite
    @Override
    public List<ComponenteHospital> getSubordinados() {
        return Collections.emptyList(); // No tiene subordinados
    }
}

```

Pruebas:

Se modificó la Vista_Principal para poder ingresar un grupo de usuarios.



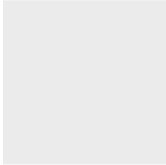
De igual manera al hacer clic en Iniciar sesion, está la opción de registrar un grupo.

Iniciar Sesión

 **ISSSTE**
INSTITUTO DE SEGURIDAD Y SERVICIOS SOCIALES DE LOS TRABAJADORES DEL ESTADO

[Inicio](#) [Equipo Médico](#) [Preguntas Frecuentes](#)

Iniciar Sesión



Correo Electronico

Contraseña

[¿No tienes una cuenta? Registrate](#) o [Registrar grupo](#)

Al momento de estar en la ventana Vista_Registro_Grupo, tendremos la opción de agregar un usuario al grupo y de registrar el grupo.

 **ISSSTE**
INSTITUTO DE SEGURIDAD Y SERVICIOS SOCIALES DE LOS TRABAJADORES DEL ESTADO

[Inicio](#) [Equipo Médico](#) [Preguntas Frecuentes](#)

Registro de usuario

Nombre

Apellido Paterno

Apellido Materno

CURP

RFC

Correo Electronico

Contraseña

Confirmar contraseña

Una vez que hemos agregado los usuarios al grupo, podremos registrar el grupo.

6. Prototype

El patrón de diseño prototipo tiene como finalidad crear nuevos objetos clonando una instancia creada previamente. Este patrón trabaja en conjunto con el patrón Composite, ya que al registrar usuarios mediante este patrón, el patrón Prototype los clona.

Funciona con las siguientes clases:

1. Interfaz Cloneable, contiene el método para clonar el objeto.
2. Clase Usuario, el objeto que se clonara.
3. Grupo Usuarios, el objeto que se clonara.
4. Clase Medico, objeto que se clonara.

Interfaz Cloneable:

```
public interface Cloneable {
    // No contiene métodos. Es una interfaz de marcador.
}
```

Clase Usuario:

```
public class Usuario implements Cloneable, ComponenteHospital{
    private String rfc, curp, correo, contraseña, nombre, apellidoMaterno,
    apellidoPaterno, tipo;
    private int id;
    public int getId() {
        return id;
    }

    public void setId(int id) {
```



```

        this.id = id;
    }

    public void setTipo(String tipo) {
        this.tipo = tipo;
    }

    public Usuario(String rfc, String curp, String correo, String contraseña, String
nombre, String apellidoMaterno, String apellidoPaterno) {
        this.rfc = rfc;
        this.curp = curp;
        this.correo = correo;
        this.contraseña = contraseña;
        this.nombre = nombre;
        this.apellidoMaterno = apellidoMaterno;
        this.apellidoPaterno = apellidoPaterno;
    }

    public String getTipo() {
        return tipo;
    }

    public String getRfc() {
        return rfc;
    }

    public String getCurp() {
        return curp;
    }

    public String getCorreo() {
        return correo;
    }

    public String getContraseña() {
        return contraseña;
    }

    public String getNombre() {
        return nombre;
    }

    public String getApellidoMaterno() {
        return apellidoMaterno;
    }

    public String getApellidoPaterno() {
        return apellidoPaterno;
    }

    public void setRfc(String rfc) {
        this.rfc = rfc;
    }

    public void setCurp(String curp) {
        this.curp = curp;
    }

```

```

    }

    public void setCorreo(String correo) {
        this.correo = correo;
    }

    public void setContraseña(String contraseña) {
        this.contraseña = contraseña;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public void setApellidoMaterno(String apellidoMaterno) {
        this.apellidoMaterno = apellidoMaterno;
    }

    public void setApellidoPaterno(String apellidoPaterno) {
        this.apellidoPaterno = apellidoPaterno;
    }

    // Implementación del patrón Prototype
    @Override
    public Usuario clone() {
        try {
            Usuario clon = (Usuario) super.clone();
            clon.setId(0); // Para que la base de datos asigne un nuevo ID

            // Modificar correo para evitar conflictos en la base de datos
            clon.setCorreo(this.correo.replace("@", "+clon@"));

            // Modificar RFC y CURP (opcional)
            clon.setRfc(this.rfc + "C");
            clon.setCarp(this.carp + "C");

            return clon;
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException("Error al clonar Usuario", e);
        }
    }

    //implementación del patrón composite
    @Override
    public void mostrarInformacion() {
        System.out.println("Usuario: " + nombre + " " + apellidoPaterno + " - RFC: "
+ rfc);
    }

    //implementación del patrón composite
    @Override
    public void agregar(ComponenteHospital componente) {
        throw new UnsupportedOperationException("No aplicable para usuarios
individuales");
    }

```

```

        //implementación del patrón composite
        @Override
        public void eliminar(ComponenteHospital componente) {
            throw new UnsupportedOperationException("No aplicable para usuarios individuales");
        }

        //implementación del patrón composite
        @Override
        public List<ComponenteHospital> getSubordinados() {
            return Collections.emptyList(); // No tiene subordinados
        }
    }
}

```

Clase GrupoUsuarios:

```

public class GrupoUsuarios {
    private List<Usuario> usuarios;
    private Connection conn; // Conexión a la base de datos

    // Constructor
    public GrupoUsuarios(Connection conn) {
        this.usuarios = new ArrayList<>();
        this.conn = conn;
    }

    // Agregar usuario al grupo
    public void agregarUsuario(Usuario usuario) {
        if (usuario != null) {
            usuarios.add(usuario);
        }
    }

    // Eliminar usuario del grupo
    public void eliminarUsuario(Usuario usuario) {
        usuarios.remove(usuario);
    }

    // Registrar todos los usuarios del grupo
    public boolean registrarUsuarios() {
        if (usuarios.isEmpty()) {
            return false;
        }

        try {
            for (Usuario usuario : usuarios) {
                RegistrarUsuario.registrar(conn, usuario);
            }
            usuarios.clear(); // Limpiar la lista después de registrar
            return true;
        } catch (SQLException ex) {
            ex.printStackTrace();
            return false;
        }
    }
}

```

```

// Obtener la lista de usuarios
public List<Usuario> getUsuarios() {
    return usuarios;
}

// Verificar si el grupo está vacío
public boolean estaVacio() {
    return usuarios.isEmpty();
}
}

```

Clase Medico:

```

public class Medico implements Cloneable, ComponenteHospital{
    private int id;
    private String nombre;
    private String especialidad;
    private String correo;
    public Medico(int id, String nombre, String especialidad, String correo) {
        this.id = id;
        this.nombre = nombre;
        this.especialidad = especialidad;
        this.correo=correo;
    }

    public int getId() {
        return id;
    }

    public String getCorreo() {
        return correo;
    }

    public void setCorreo(String correo) {
        this.correo = correo;
    }

    public String getNombre() {
        return nombre;
    }

    public String getEspecialidad() {
        return especialidad;
    }

    // Implementación del patrón Prototype
    @Override
    public Medico clone() {
        try {
            return (Medico) super.clone(); // Clonación superficial
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException("Error al clonar Medico", e);
        }
    }
}

```

```

//implementación del patrón composite
@Override
public void mostrarInformacion() {
    System.out.println("Médico: " + nombre + " - Especialidad: " + especialidad);
}

//implementación del patrón composite
@Override
public void agregar(ComponenteHospital componente) {
    throw new UnsupportedOperationException("No aplicable para usuarios
individuales");
}

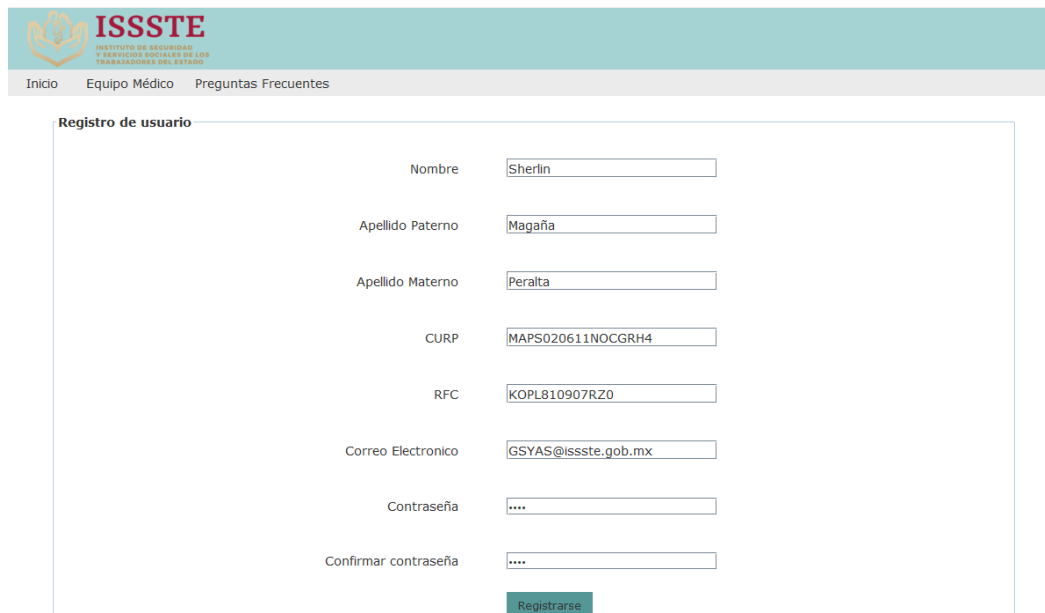
//implementación del patrón composite
@Override
public void eliminar(ComponenteHospital componente) {
    throw new UnsupportedOperationException("No aplicable para usuarios
individuales");
}

//implementación del patrón composite
@Override
public List<ComponenteHospital> getSubordinados() {
    return Collections.emptyList(); // No tiene subordinados
}
}

```

Pruebas:

Al momento de registrar un usuario, veremos que este usuario se clona con id autoincremental y los mismos datos que ya establecimos para el usuario a registrar.



ISSSTE
INSTITUTO DE SEGURIDAD Y SERVICIOS SOCIALES DE LOS TRABAJADORES DEL ESTADO

Inicio Equipo Médico Preguntas Frecuentes

Registro de usuario

Nombre

Apellido Paterno

Apellido Materno

CURP

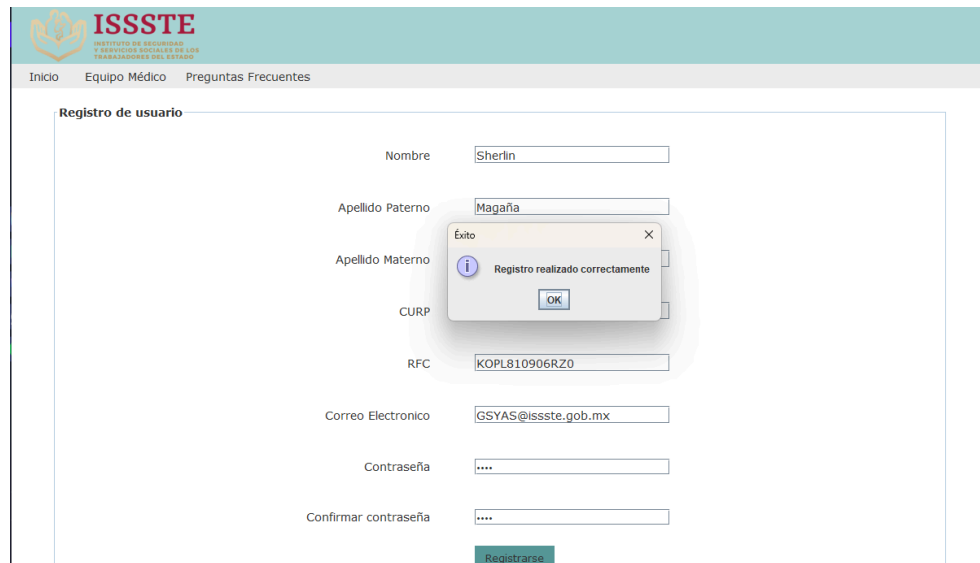
RFC

Correo Electronico

Contraseña

Confirmar contraseña

Usuario original registrado:



The screenshot shows the ISSSTE (Instituto de Seguridad y Servicios Sociales de los Trabajadores del Estado) user registration interface. The form is titled 'Registro de usuario' and contains the following fields: Nombre (Sherlin), Apellido Paterno (Magaña), Apellido Materno, CURP, RFC (KOPL810906RZ0), Correo Electronico (GSYAS@issste.gob.mx), Contraseña, and Confirmar contraseña. A modal dialog box with the title 'Éxito' and a close button 'X' is displayed in the center, containing the message 'Registro realizado correctamente' and an 'OK' button. The background is a light blue header with the ISSSTE logo and navigation links: Inicio, Equipo Médico, and Preguntas Frecuentes.

Usuario clonado registrado:

`Usuario clonado registrado correctamente.`

7. Proxy

Un proxy controla el acceso al objeto original, permitiéndote hacer algo antes o después de que la solicitud llegue al objeto original.

En el proyecto, este patrón funciona en conjunto con patrón Facade por que al iniciar sesión el usuario, manda solicitud al Proxy y redirige mediante el Facade.

Este patrón funciona mediante las siguientes clases:

1. Interfaz IRegistroUsuario
2. Clase Usuario, clase original del objeto que de registrara.
3. Clase RegistroUsuarioReal, clase que realiza el registro real.
4. Clase RegistroUsuarioProxy, verifica si el usuario tiene permisos para registrarse antes de llamar al real.

Interfaz IRegistroUsuario:

```
public interface IRegistroUsuario {  
    boolean registrar(Usuario usuario) throws SQLException;  
}
```

Clase Usuario:

```

public class Usuario implements Cloneable, ComponenteHospital{
    private String rfc, curp, correo, contraseña, nombre, apellidoMaterno,
apellidoPaterno, tipo;
    private int id;
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public void setTipo(String tipo) {
        this.tipo = tipo;
    }

    public Usuario(String rfc, String curp, String correo, String contraseña, String
nombre, String apellidoMaterno, String apellidoPaterno) {
        this.rfc = rfc;
        this.curp = curp;
        this.correo = correo;
        this.contraseña = contraseña;
        this.nombre = nombre;
        this.apellidoMaterno = apellidoMaterno;
        this.apellidoPaterno = apellidoPaterno;
    }

    public String getTipo() {
        return tipo;
    }

    public String getRfc() {
        return rfc;
    }

    public String getCurp() {
        return curp;
    }

    public String getCorreo() {
        return correo;
    }

    public String getContraseña() {
        return contraseña;
    }

    public String getNombre() {
        return nombre;
    }

    public String getApellidoMaterno() {
        return apellidoMaterno;
    }

    public String getApellidoPaterno() {

```

```

        return apellidoPaterno;
    }

    public void setRfc(String rfc) {
        this.rfc = rfc;
    }

    public void setCurp(String curp) {
        this.curp = curp;
    }

    public void setCorreo(String correo) {
        this.correo = correo;
    }

    public void setContraseña(String contraseña) {
        this.contraseña = contraseña;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public void setApellidoMaterno(String apellidoMaterno) {
        this.apellidoMaterno = apellidoMaterno;
    }

    public void setApellidoPaterno(String apellidoPaterno) {
        this.apellidoPaterno = apellidoPaterno;
    }

    // Implementación del patrón Prototype
    @Override
    public Usuario clone() {
        try {
            Usuario clon = (Usuario) super.clone();
            clon.setId(0); // Para que la base de datos asigne un nuevo ID

            // Modificar correo para evitar conflictos en la base de datos
            clon.setCorreo(this.correo.replace("@", "+clon@"));

            // Modificar RFC y CURP (opcional)
            clon.setRfc(this.rfc + "C");
            clon.setCurp(this.curp + "C");

            return clon;
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException("Error al clonar Usuario", e);
        }
    }

    //implementación del patrón composite
    @Override
    public void mostrarInformacion() {
        System.out.println("Usuario: " + nombre + " " + apellidoPaterno + " - RFC: "
+ rfc);
    }

```



```

    }

    //implementación del patrón composite
    @Override
    public void agregar(ComponenteHospital componente) {
        throw new UnsupportedOperationException("No aplicable para usuarios
individuales");
    }

    //implementación del patrón composite
    @Override
    public void eliminar(ComponenteHospital componente) {
        throw new UnsupportedOperationException("No aplicable para usuarios
individuales");
    }

    //implementación del patrón composite
    @Override
    public List<ComponenteHospital> getSubordinados() {
        return Collections.emptyList(); // No tiene subordinados
    }
}

```

Clase RegistroUsuarioReal:

```

public class RegistroUsuarioReal implements IRegistroUsuario {
    private Connection conn;

    public RegistroUsuarioReal(Connection conn) {
        this.conn = conn;
    }

    @Override
    public boolean registrar(Usuario usuario) throws SQLException {
        return RegistrarUsuario.registrar(conn, usuario);
    }
}

```

Clase RegistroUsuarioProxy:

```

public class RegistroUsuarioProxy implements IRegistroUsuario {
    private IRegistroUsuario registroReal;

    public RegistroUsuarioProxy(IRegistroUsuario registroReal) {
        this.registroReal = registroReal;
    }

    @Override
    public boolean registrar(Usuario usuario) throws SQLException {
        if (!usuario.getCorreo().endsWith("@issste.gob.mx")) {
            JOptionPane.showMessageDialog(null, "El correo debe ser institucional
(\"@issste.gob.mx\")");
            return false;
        }
    }
}

```

```
    return registroReal.registrar(usuario);  
  }  
}
```

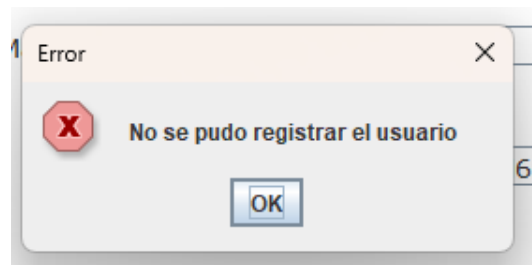
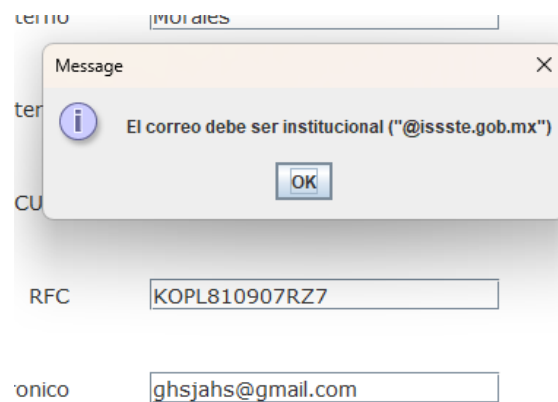
Pruebas:

Al intentar registrar un usuario, enviaremos una solicitud al proxy el cual tiene una restricción de que solo se pueden registrar correos institucionales.

Por lo tanto al ingresar otro tipo de correo, nos rechazara la solicitud de registro.

Registro de usuario

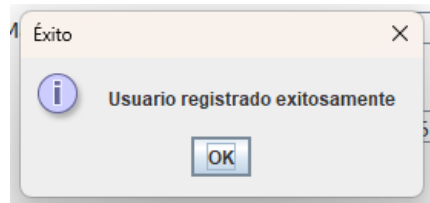
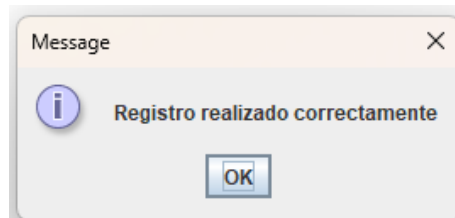
Nombre	Luis
Apellido Paterno	Morales
Apellido Materno	Lopez
CURP	JAAD651123POCDYSM6
RFC	KOPL810907RZ7
Correo Electronico	ghsjahs@gmail.com
Contraseña	
Confirmar contraseña	
Registrarse	



Entonces al hacer un uso de un correo institucional, la solicitud sera aceptada y el usuario se registrará correctamente.

Registro de usuario

Nombre	Luis
Apellido Paterno	Morales
Apellido Materno	Lopez
CURP	JAAD651123POCDYSM6
RFC	KOPL810907RZ7
Correo Electronico	ghsjahs@issste.gob.mx
Contraseña	
Confirmar contraseña	



8. Bridge

El código sigue la estructura del Patrón Bridge, donde FormatoCita es la abstracción y FormatoCitaBD es la implementación que creamos.

Interfaz FormatoCitaImplementacion

Define los métodos que cualquier implementación de FormatoCita debe seguir.

```
public interface FormatoCitaImplementacion {  
  
    void obtenerDatosSucursales(Connection conn, JComboBox<String> sucursalComboBox);  
  
    void devolverIDMConnection conn, String idMedico);
```

```

void obtenerDatosMedicos(Connection conn, JComboBox<String> medico
sComboBox);

void obtenerFechasDisponibles(Connection conn, JComboBox<String> fec
hasComboBox);

void obtenerHorasDisponibles(Connection conn, JComboBox<String> hora
sComboBox);

void devolverConsultorio(Connection conn, String idMedico);

void cambiarEstadoCita(Connection conn, String idCita, String estado);

String obtenerIdCita(Connection conn);

}

```

Clase FormatoCita (Abstracción)

Esta clase mantiene una referencia a FormatoCitaImplementacion , permitiendo que las solicitudes de los clientes sean delegadas a la implementación específica.

```

/*
 * To change this license header, choose License Headers in Project Propertie
s.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
/**
 *
 * @author gatito-asesino
 */
public class FormatoCita {

protected FormatoCitaImplementacion implementacion;

```

```

public FormatoCita(FormatoCitaImplementacion implementacion) {
    this.implementacion = implementacion;
}

public void obtenerSucursales(Connection conn, JComboBox<String> sucu
rsalComboBox) {
    implementacion.obtenerDatosSucursales(conn, sucursalComboBox);
}

public void obtenerMedicos(Connection conn, JComboBox<String> medico
sComboBox) {
    implementacion.obtenerDatosMedicos(conn, medicosComboBox);
}

public void obtenerFechas(Connection conn, JComboBox<String> fechasC
omboBox) {
    implementacion.obtenerFechasDisponibles(conn, fechasComboBox);
}

public void obtenerHoras(Connection conn, JComboBox<String> horasCo
mboBox) {
    implementacion.obtenerHorasDisponibles(conn, horasComboBox);
}
}

```

Clase FormatoCitaBD (Implementación)

Proporciona la lógica concreta para acceder a los datos de la base de datos.

```
/*
```

```

 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.t
xt to change this license

```

* Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit
this template

```
*/  
  
package Modelo;  
  
import javax.swing.*;  
  
import java.sql.*;  
  
import java.util.ArrayList;  
  
/**  
  
*  
  
* @author crist  
  
*/  
  
public class Formato_CitasBD  
  
// ArrayLists para almacenar los datos de la base de datos  
  
private static ArrayList<Integer> idSucursales = new ArrayList<>();  
  
private static ArrayList<Integer> idMedicos = new ArrayList<>();  
  
private static ArrayList<Date> fechas = new ArrayList<>();  
  
private static ArrayList<Time> horas = new ArrayList<>();  
  
// Método para obtener los datos de las sucursales desde la base de datos  
  
public static void obtenerDatosSucursales(Connection conn, JComboBox<  
String> sucursalComboBox) {  
  
try {  
  
String query = "SELECT id, entidad FROM Sucursales";  
  
Statement stmt = conn.createStatement();  
  
ResultSet rs = stmt.executeQuery(query);
```

```

while (rs.next()) {

int id = rs.getInt("id");

String entidad = rs.getString("entidad");

idSucursales.add(id);

sucursalComboBox.addItem(entidad);

}

stmt.close();

} catch SQLException e) {

e.printStackTrace();

}

}

public static int devolverIDMConnection conn, String nombre) {

int nombre11;

try {

String query = "SELECT id FROM Especialistas WHERE nombre = '"+n
ombre+"'";

Statement stmt = conn.createStatement();

ResultSet rs = stmt.executeQuery(query);

while (rs.next()) {

nombre1 rs.getInt("id");

}

} catch SQLException e) {

e.printStackTrace();

}

```

```

return nombre1;
}

// Método para obtener los datos de los médicos desde la base de datos
public static void obtenerDatosMedicos(Connection conn, JComboBox<String> medicoComboBox, int Sucursal, String tipo) {
    try {
        String query = "SELECT distinct nombre FROM Especialistas WHERE especialidad = '"+tipo+"'";
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(query);
        while (rs.next()) {
            String nombre = rs.getString("nombre");
            medicoComboBox.addItem(nombre);
        }
        stmt.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

// Método para obtener las fechas disponibles desde la base de datos
public static void obtenerFechasDisponibles(Connection conn, JComboBox<String> fechaComboBox, int idMedico) {
    try {
        System.out.println(idMedico);
        String query = "SELECT DISTINCT fecha FROM Citas WHERE id_medico

```



```

? AND disponible TRUE";

PreparedStatement stmt = conn.prepareStatement(query);

stmt.setInt(1, idMedico);

ResultSet rs = stmt.executeQuery();

while (rs.next()) {

    System.out.println("con");

    Date fecha = rs.getDate("fecha");

    fechaComboBox.addItem(fecha.toString());

}

stmt.close();

} catch SQLException e) {

    e.printStackTrace();

}

}

// Método para obtener las horas disponibles desde la base de datos

public static void obtenerHorasDisponibles(Connection conn, JComboBox<

String> horaComboBox, String fecha, int idMedico) {

    try {

        String query = "SELECT DISTINCT hora FROM Citas WHERE fecha ? A

ND id_medico ? AND disponible TRUE";

        PreparedStatement stmt = conn.prepareStatement(query);

        stmt.setString(1, fecha);

        stmt.setInt(2, idMedico);

        ResultSet rs = stmt.executeQuery();

```

```
while (rs.next()) {  
  
    Time hora = rs.getTime("hora");  
  
    horaComboBox.addItem(hora.toString());  
  
}  
  
stmt.close();  
  
} catch SQLException e) {  
  
    e.printStackTrace();  
  
}  
  
}
```

```
public static String devolverConsultorio(Connection conn, String id) {  
  
    String nombre1=null;  
  
    try {  
  
        String query = "SELECT consultorio FROM Especialistas WHERE id =  
        '"+id+"'";  
  
        Statement stmt = conn.createStatement();  
  
        ResultSet rs = stmt.executeQuery(query);  
  
        while (rs.next()) {  
  
            nombre1 rs.getString("consultorio");  
  
        }  
  
        } catch SQLException e) {  
  
            e.printStackTrace();  
  
        }  
  
        return nombre1;  
  
    }
```

```

public static void cambiarEstadoCita(Connection conn, int idCita, int idAfiliado, String nombreAfiliado, String fechaAgendada, String horaAgendada, String doctorAsignado, String fechaCita, String horaCita, String consultorio, String tipo, int idMedico) {
    try {
        boolean captura;

        captura = ControladorVista_Historial_De_Citas.ultimaCitaYaPaso(conn, tipo, idAfiliado+"");

        if(captura){
            String query = "UPDATE Citas SET disponible = FALSE, id_paciente = ? WHERE id = ?";

            PreparedStatement pstmt = conn.prepareStatement(query);

            pstmt.setInt(1, idAfiliado);

            pstmt.setInt(2, idCita);

            pstmt.executeUpdate();

            pstmt.close();

            System.out.println(tipo);

            // Ahora insertamos los datos en la tabla citas_afiliado

            String insertQuery = "INSERT INTO citas_afiliado (id_cita, id_paciente, nombre_paciente, fecha_agendada, hora_agendada, doctor_asignado, fecha_cita, hora_cita, consultorio, tipo, id_medico) VALUES ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?";

            PreparedStatement insertStmt = conn.prepareStatement(insertQuery);

            insertStmt.setInt(1, idCita);

            insertStmt.setInt(2, idAfiliado);

```

```

insertStmt.setString(3, nombreAfiliado);
insertStmt.setString(4, fechaAgendada);
insertStmt.setString(5, horaAgendada);
insertStmt.setString(6, doctorAsignado);
insertStmt.setString(7, fechaCita);
insertStmt.setString(8, horaCita);
insertStmt.setString(9, consultorio);
insertStmt.setString(10, tipo);
insertStmt.setInt(11, idMedico);
insertStmt.executeUpdate();
insertStmt.close();

JOptionPane.showMessageDialog(null, "La cita ha sido agendada correctamente");}

else{

JOptionPane.showMessageDialog(null, "Tienes una cita agendada aún, intentalo de nuevo despues de finalizar tu cita");

}

} catch SQLException e) {

e.printStackTrace();

}

}

public static int obtenerIdCita(Connection conn, int idDoctor, String fecha, String hora)
{https://drive.google.com/drive/folders/1gf-uPnxSuhmf5vlnioRJnkLrjIE0e7Ht?usp=sharing

int idCita = 1;

https://drive.google.com/drive/folders/1gf-uPnxSuhmf5vlnioRJnkLrjIE0e7Ht?usp=sharing
try {

```

```

String query = "SELECT id FROM Citas WHERE id_medico ? AND fecha
? AND hora ? AND disponible TRUE";

PreparedStatement pstmt = conn.prepareStatement(query);

pstmt.setInt(1, idDoctor);

pstmt.setString(2, fecha);


pstmt.setString(3, hora);

ResultSet rs = pstmt.executeQuery();

if (rs.next()) {

idCita = rs.getInt("id");

}

pstmt.close();

} catch SQLException e) {

e.printStackTrace();

}

return idCita;

}

}

```

Uso del código

Finalmente, para utilizar el código, se instancia la abstracción con una implementación específica:

```

FormatoCitaImplementacion formatoCita = new FormatoCitaBD;

FormatoCita cita = new FormatoCita(formatoCita);

cita.obtenerSucursales(conn, sucursalComboBox);

cita.obtenerMedicos(conn, medicosComboBox);

cita.obtenerFechas(conn, fechasComboBox);

```

```
cita.obtenerHoras(conn, horasComboBox);
```

9. Factory

El Patrón Factory ha sido una excelente elección para nuestro sistema de citas médicas, ya que nos permite manejar la creación de diferentes tipos de usuarios de manera modular y escalable. Gracias a este patrón, el sistema es más fácil de mantener y extender en el futuro.

Clase PersoanFactory

```
package Modelo;

import Clases.Usuario;

import Clases.Medico;

public class PersonaFactory {

    public static Object crearPersona(String tipo, String[] datos) {

Implementación del Patrón Factory en el Sistema de Citas Médicas 3

        switch (tipo.toLowerCase()) {

            case "usuario":

                return new Usuario(datos[0], datos[1], datos[2], datos[3], datos[4],

dato

            case "medico":

                return new Medico(Integer.parseInt(datos[0]), datos[1], datos[2],

datos

            default:

                throw new IllegalArgumentException("Tipo de persona no válido: " +

tip

        }

    }

}
```

Clase Usuario

```
/*
```

```
* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt
```

* Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit thi

*/

package Clases;

/**

*

* @author crist

*/

public class Usuario {

private String rfc, curp, correo, contraseña, nombre, apellidoMaterno, apellido

private int id;

public int getId() {

return id;

}

public void setId(int id) {

this.id = id;

}

4

Implementación del Patrón Factory en el Sistema de Citas Médicas

public void setTipo(String tipo) {

this.tipo = tipo;

}

public Usuario(String rfc, String curp, String correo, String contraseña, String

n

this.rfc = rfc;

this.curp = curp;

this.correo = correo;

```

        this.contraseña = contraseña;

        this.nombre = nombre;

        this.apellidoMaterno = apellidoMaterno;

        this.apellidoPaterno = apellidoPaterno;
    }

    public String getTipo() {

        return tipo;
    }

    public String getRfc() {

        return rfc;
    }

    public String getCurp() {

        return curp;
    }

    public String getCorreo() {

        return correo;
    }

    public String getContraseña() {

        return contraseña;
    }

    public String getNombre() {

```

5

Implementación del Patrón Factory en el Sistema de Citas Médicas

```

        return nombre;
    }

```



```
public String getApellidoMaterno() {  
    return apellidoMaterno;  
}  
  
public String getApellidoPaterno() {  
    return apellidoPaterno;  
}  
  
public void setRfc(String rfc) {  
    this.rfc = rfc;  
}  
  
public void setCurp(String curp) {  
    this.curp = curp;  
}  
  
public void setCorreo(String correo) {  
    this.correo = correo;  
}  
  
public void setContraseña(String contraseña) {  
    this.contraseña = contraseña;  
}  
  
public void setNombre(String nombre) {  
    this.nombre = nombre;  
}  
  
public void setApellidoMaterno(String apellidoMaterno) {  
    this.apellidoMaterno = apellidoMaterno;  
}  
  
public void setApellidoPaterno(String apellidoPaterno) {
```

Implementación del Patrón Factory en el Sistema de Citas Médicas

```
        this.apellidoPaterno = apellidoPaterno;
    }
}

Clase Médico

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit thi
 */
package Clases;

public class Medico {
    private int id;
    private String nombre;
    private String especialidad;
    private String correo;

    public Medico(int id, String nombre, String especialidad, String correo) {
        this.id = id;
        this.nombre = nombre;
        this.especialidad = especialidad;
        this.correo = correo;
    }

    public int getId() {
        return id;
    }
}
```

```

public String getCorreo() {
    return correo;
}

```

7

Implementación del Patrón Factory en el Sistema de Citas Médicas

```

public void setCorreo(String correo) {
    this.correo = correo;
}

public String getNombre() {
    return nombre;
}

public String getEspecialidad() {
    return especialidad;
}

```

Override

```

public String toString() {
    return nombre + " (" + especialidad + ")";
}
}

```

Implementación en los botones mouseClicked

```

private void RegistrarseMouseClicked(java.awt.event.MouseEvent evt) {GENF
    // TODO add your handling code here:

    String rfcText = rfc.getText();

    String curpText = curp.getText();

    String Nombre = nombre.getText();

```

```

String correoText = correo.getText();

String contraseñaText = new String(contraseña.getPassword());

String confirmarContraseñaText = new String(confirmar_contraseña.getPass

String apellidoMaternoText = apellido_m.getText();

String apellidoPaternoText = apellido_p.getText();

if (validarCampos(Nombre, correoText, contraseñaText, confirmarContraseñ

    if (validarCurp(curpText) && validarRFC(rfcText)) {

        if

!contraseñaText.equals(confirmarContraseñaText)) {

            JOptionPane.showMessageDialog(this, "Las contraseñas no coincide

8

Implementación del Patrón Factory en el Sistema de Citas Médicas

        return;

    }

    // Crear el objeto Usuario usando la fábrica

String[] datosUsuario = {rfcText, curpText, correoText,
contraseñaText,

    Usuario usuario = Usuario) PersonaFactory.crearPersona("usuario", da

    try {

        if

RegistrarUsuario.registrar(conn, usuario)) {

            Iniciar_Sesion iniciar = new Iniciar_Sesion(conn);

            iniciar.setVisible(true);

            this.setVisible(false);

        }

    } catch SQLException ex) {

```

```

        Logger.getLogger(Vista_Registro_Usuario.class.getName()).log(Leve
    }
}
}
}

private void confirmar_contraseñaActionPerformed(java.awt.event.ActionEvent e) {
    String rfcText = rfc.getText();
    String curpText = curp.getText();
    String Nombre = nombre.getText();
    String correoText = correo.getText();
    String contraseñaText = new String(contraseña.getPassword());
    String confirmarContraseñaText = new String(confirmar_contraseña.getPass
    String apellidoMaternoText = apellido_m.getText();
    String apellidoPaternoText = apellido_p.getText();

    if
Nombre.length() == 0 || correoText.length() == 0 || contraseñaText.length
        JOptionPane.showMessageDialog(this, "Rellena todos los campos", "Erro
        return;
    }
}

```

9

Implementación del Patrón Factory en el Sistema de Citas Médicas

```

        if (validarCurp(curpText) && validarRFC(rfcText)) {
            if
        }

        !contraseñaText.equals(confirmarContraseñaText)) {

```

```

        JOptionPane.showMessageDialog(this, "Las contraseñas no coinciden"

        return;

        // Crear el objeto Usuario si la contraseña coincide

        String[] datosUsuario = {rfcText, curpText, correoText, contraseñaText,

No
        Usuario usuario = Usuario) PersonaFactory.crearPersona("usuario", dato

        try {

            if

RegistrarUsuario.registrar(conn, usuario)) {

                Iniciar_Sesion iniciar = new Iniciar_Sesion(conn);

                iniciar.setVisible(true);

                this.setVisible(false);

            }

        } catch SQLException ex) {

            Logger.getLogger(Vista_Registro_Usuario.class.getName()).log(Level.S

        }

    }

} //GENLAST:event_con

```

Pruebas

Al momento de registrar un nuevo usuario, se crea una persona para delegar el trabajo.

Registro Usuario

 **ISSSTE**
INSTITUTO DE SEGURIDAD
Y SERVICIOS SOCIALES DE LOS
TRABAJADORES DEL ESTADO

[Inicio](#) [Equipo Médico](#) [Preguntas Frecuentes](#)

Registro de usuario

Nombre

Apellido Paterno

Apellido Materno

CURP

RFC

Correo Electronico

Contraseña

Confirmar contraseña

¿Ya tienes una cuenta?, [Inicia Sesión](#)

Dudas y contacto: atencionciudadana@issste.gob.mx o al 55-4000-1000

Referencias

1. Oracle. (n.d.). **Java Swing Documentation**. Recuperado el 11 de junio de 2024, de <https://docs.oracle.com/javase/tutorial/uiswing/>
2. Oracle. (n.d.). **JDBC Documentation**. Recuperado el 11 de junio de 2024, de <https://docs.oracle.com/javase/tutorial/jdbc/>
3. Oracle. (n.d.). **MySQL Documentation**. Recuperado el 11 de junio de 2024, de <https://dev.mysql.com/doc/>
4. Oracle. (n.d.). **Java SE 8 API Documentation**. Recuperado el 11 de junio de 2024, de <https://docs.oracle.com/javase/8/docs/api/>